International Journal of Intelligent Systems

WILEY

Allergy EUROPEAN JOURNAL OF ALLERGY AND CLINICAL IMMUNOLOGY EAACI

ORIGINAL ARTICLE OPEN ACCESS

An Adaptive, Closed-Loop CNN-LSTM Framework for Real-Time DoS/DDoS Detection and Mitigation in Software-Defined Networks under Concept Drift

Tahir Ali[1] | Hasan Mahmood[1] | Muddasar Naeem[2] | Asad Hussain[3] | Shumayla Yaqoob[4] | Antonio Coronato[2] | Farman Ullah[5]

[1]Department of Electronics Quaid-i-Azam University Islamabad, Pakistan | [2]Artificial Intelligence and Robotics Lab, Università Giustino Fortunato, Benevento, Italy | [3]Department of Computer Science, University of University of Reading, Reading, UK | [4]Faculty of Computing and Information Technology (FCIT) Sohar University, Sohar, Oman [5]College of Information Technology (CIT) United Arab Emirates University, Alain, UAE |

Correspondence: Farman Ullah (farman@uaeu.ac.ae)

Received: 20 January 2023 | **Revised:** 21 January 2023 | **Accepted:** 22 January 2023

Keywords: Software-defined networking | DDoS detection and mitigation | Concept drift | Online learning | Cross-dataset generalization | Hybrid CNN-LSTM

ABSTRACT

Software-Defined Networking (SDN) centralizes control to make networks easier to operate, but the same centralization turns the controller into an attractive target for Denial-of-Service and Distributed Denial-of-Service (DoS/DDoS) attacks. Learning-based detectors for this problem have been proposed in large numbers, yet most share three weaknesses that limit their value in practice: they are trained once and then frozen, so accuracy decays when traffic shifts; they stop at raising an alert and never act on the network, so the claimed real-time benefit is never measured; and they are validated on the same dataset used for training, which says little about unseen traffic. This work treats those three issues as the problem to solve. We retain a compact CNN-LSTM model, in which the convolutional layers capture spatial structure in flow records and the recurrent layers model their temporal behavior, and build three capabilities around it. First, the detector adapts online: a drift detector monitors the live error rate and triggers short incremental updates, while a replay buffer preserves earlier knowledge and limits forgetting. Second, the detector runs in closed loop with the controller, installing OpenFlow rules to drop or rate-limit malicious sources so that the cost of mitigation can be measured rather than assumed. Third, the model is evaluated across datasets, trained on CICDDoS2019 and tested on the SDN-specific InSDN dataset and on live traffic from a Mininet, POX, and Open vSwitch testbed instrumented with sFlow. To keep the results trustworthy, class balancing is confined to the training folds and performance is reported on the natural class distribution using PR-AUC, the Matthews correlation coefficient, and per-class recall alongside accuracy. On CICDDoS2019 the model, with roughly 47,000 parameters, detects attacks at 99.95% accuracy and an F1-score of 99.98%; under simulated concept drift it recovers previously unseen attack families within about a dozen incremental updates, raising prequential accuracy from 0.59 to 0.93 where a static model stalls; and, trained on CICDDoS2019, it transfers to the SDN-specific InSDN dataset, where a brief adaptation step lifts zero-shot accuracy from 0.32 to 0.97. The detector also drives a closed-loop mitigation path on the controller within a Mininet, POX, and Open vSwitch testbed, measurably reducing Packet-In load on the controller; live per-source classification did not reliably distinguish the attacking hosts from the benign one in this testbed, and we report that finding alongside the mechanism it qualifies rather than omit it.

1 | Introduction

Software-Defined Networking (SDN) is a major advancement in modern network architecture, which separates the control plane from the data plane [1].This separation allows centralized network control, simplifying configuration, automation, and management. In this architecture as shown in Figure 1, the control plane decides how the traffic flows, while the data plane handles

the actual forwarding of packets. As a result, SDN has widespread adoption in data centers, enterprise environments, and cloud infrastructures where scalability and flexibility are in high demand. [2].

Despite its benefits, the centralized architecture of SDN introduces significant security challenges. The SDN controller acts as the brain of the network [3] and becomes a single point of failure for attackers [4]. It is particularly vulnerable to DoS/DDoS attacks, which aim to exhaust system resources by flooding the controller with fake flow requests to disturb traffic volume [5]. The DoS attacks originate from a single source, and DDoS attacks harness large numbers of compromised devices across multiple locations, making them harder to detect and mitigate [4]. These attacks compromise service availability, leading to network downtime, financial loss, and credibility damage. Such attacks are also often used to divert attention from more advanced breaches. The DDoS attacks are growing exponentially, both in frequency and sophistication. From early disruptions in 1999 and 2000, the scale of attacks has escalated dramatically, driven partly by the proliferation of Internet of Things (IoT) botnets [6].

Furthermore, emergence of low-rate, hard-to-detect attacks such as Shrew DDoS further complicates the detection efforts within SDN environments [7]. The volume of IoT traffic continues to rise, and the AI detection mechanism is showing promise in detecting these sophisticated threats, making them a key focus in the development of resilient, SDN-aware security solutions [8]. The evolution of DDoS attacks over the past two decades reflects a significant increase in scale, automation, and technical sophistication as shown in Table 1.These statistics highlight the growing complexity of modern threats in SDN environments.

The limitations of current DDoS detection approaches in SDN highlight the need for more robust solutions. Static rule-based systems use fixed rules and cannot adopt new attack patterns [9]. Traditional machine learning (ML) methods often suffer from high false positive rates (FPR) and lack real-time scalability [10]. Many ML models, such as random forest (RF), support vector machines (SVM), decision tree (DT), logistic regression (LR), naive bayes (NB), and k-Nearest neighbors (K-NN), are evaluated solely in simulated environments [11] or with outdated datasets [12]. These datasets often lack diversity and do not reflect the latest attack strategies or real-time traffic variations. These gaps emphasize the need for adaptive and lightweight solutions tested in real-world environments.

A large body of work already applies machine learning and deep learning to DDoS detection in SDN, and much of it reports near-perfect accuracy. Three weaknesses, however, keep recurring once these models are taken seriously as something to deploy. The first is that they are trained once, offline, and then left unchanged. When the traffic mix moves or a new attack variant appears, accuracy slips quietly until an operator notices and retrains the model by hand. The second is that the pipeline usually ends at the moment of classification. A model announces that an attack is in progress but does nothing to the network, so the real-time advantage that is so often claimed is never actually measured on the control plane. The third is evaluation: training and testing are drawn from the same dataset, which reveals little about how a detector copes with traffic it has not seen.

This paper is built around those three gaps rather than around the network architecture. We keep a compact CNN-LSTM model of roughly 47,000 parameters because it is inexpensive to run next to a controller and because its convolutional and recurrent halves are a natural fit for the spatial and temporal structure of flow records. The contribution is what we wrap around it: the model is allowed to adapt to drift while it runs, it is connected to a mitigation path inside the SDN control plane so that its decisions take effect, and it is tested on data that did not come from its training set.

The evaluation is organized to test these three claims rather than a single accuracy number. The model is first trained on CICDDoS2019 [13], a benchmark that spans modern DDoS types across several protocols, including HTTP flood, UDP flood, TCP SYN flood, DNS amplification, and LDAP reflection. Generalization is then probed by testing the trained model on the SDN-specific InSDN dataset and on live captures from an emulated testbed, so that training and evaluation no longer come from the same source. The testbed itself uses Mininet with a POX controller, Open vSwitch for forwarding, and sFlow for traffic export, and it is where the closed-loop behavior is measured: on a positive detection the controller installs OpenFlow rules to drop or rate-limit the offending source. Concept drift is studied by presenting attack families as a stream and comparing a static model against the adaptive one. Throughout, class balancing with SMOTE and random undersampling [14] is applied only inside the training folds to avoid leakage into the test set, and adaptive learning-rate scheduling and early stopping are used during training.

On CICDDoS2019 the detector separates benign from malicious traffic with high accuracy at a low false positive rate, and reported on the natural class distribution rather than a rebalanced test set. More importantly for the argument of this paper, the adaptive variant recovers after drift where a frozen model loses accuracy, the model retains useful performance when moved to InSDN and to live testbed traffic, and the closed-loop defense reduces the controller's `Packet-In` load shortly after an attack begins while leaving legitimate flows largely undisturbed. Exact figures are reported in Section 5.

The main contributions of this work are as follows:

- We make the detector adapt online. A drift detector watches the prediction error on the live stream and triggers short incremental updates to the CNN-LSTM model, while a replay buffer of earlier flows guards against catastrophic forgetting. This removes the manual-retraining assumption that underlies most prior SDN detectors and lets the model track changing traffic without being rebuilt from scratch.
- We close the loop between detection and the network. Once a source is flagged, the POX controller installs an OpenFlow rule to drop it, and we report the operational cost of doing so: controller load, the drop in `Packet-In` messages, and the classification-to-rule-install latency, on a real testbed rather than assumed. This turns the common "real-time" claim into measured behavior on the control plane; whether the mechanism reliably targets the attacker rather than legitimate traffic is a separate, open finding that we report rather than assume (Section 5.8).
- We test generalization across datasets instead of within one. The model is trained on CICDDoS2019 and evaluated on the SDN-specific InSDN dataset and on live testbed captures, both with and without a brief adaptation step, which shows how well it transfers to traffic it was not trained on.

TABLE 1 | Evolution of Major DDoS Attacks and Emerging Threat Trends

Year	Incident or Campaign	Attack Characteristics	Target and Impact	Technical Significance
2007	Estonia Cyberattacks	Coordinated large-scale botnet-based DDoS	Government, banking, and media disruption	Early example of state-level cyber warfare
2012	Operation Ababil	Sustained application-layer DDoS campaign	Major U.S. financial institutions	Prolonged financial-sector service disruption
2013	Spamhaus DNS Amplification	Reflection/amplification attack (300 Gbps)	Anti-spam organization infrastructure	Rise of DNS amplification techniques
2016	Mirai Botnet Attack	IoT-driven volumetric DNS-targeted attack	Dyn DNS provider; global websites offline	Turning point toward IoT-based DDoS
2018	GitHub Memcached Attack	1.35 Tbps amplification attack	GitHub platform disruption	First terabit-scale DDoS attack
2020	COVID-19 Era Surge	Increased reflection and multi-vector attacks	Cloud platforms and remote services	Expanded digital attack surface
2023	HTTP/2 Hyper-Volumetric Attack	201 million requests per second (RPS)	Global web infrastructure	Record-breaking application-layer DDoS
2024	3.45 Tbps Mega Attack	Multi-vector terabit-scale attack	Enterprises worldwide	Largest reported bandwidth-based DDoS
2025	AI-Enhanced DDoS Campaigns	Adaptive botnets using automated traffic patterns	Cloud-native infrastructures	Emergence of AI-assisted attack strategies
2026	IoT-Edge Distributed Attacks	Encrypted, ultra-distributed edge-based traffic	SDN/5G environments	Shift toward low-latency, encrypted attack vectors

- We rebuild the evaluation protocol to remove flaws that inflate reported results. Class balancing is confined to the training folds, performance is reported on the natural (imbalanced) test distribution using PR-AUC, the Matthews correlation coefficient, and per-class recall in addition to accuracy, and the model is compared against deep-learning baselines re-implemented under identical splits with their parameter counts and inference latency reported.

The rest of the paper is organized as follows Section II Related work. Section III shows the proposed CNN-LSTM model, the description of the dataset, features selection, and training approach. The experimental set up and SDN testbed structure are described in Section IV. Section V is a discussion on the results and performance evaluation. Finally, Section VI provides the conclusion.

2 | Related Work

Over the past decade, the landscape of Internet-connected devices and cloud-based infrastructure has contributed to a significant rise in Distributed Denial-of-Service (DDoS) attacks. These attacks pose severe threats to SDN environments. The centralized control and programmability of SDN offer enhanced flexibility and scalability, but also introduce vulnerabilities [15]. In response, researchers have explored a wide range of detection techniques, ranging from traditional ML models to advanced DL and hybrid architectures, to effectively overcome these threats. Despite substantial progress, key challenges remain in achieving real-time detection, lightweight deployment, and robust generalization across diverse traffic patterns. This related work presents a structured explanation of DDoS detection strategies in SDN, organized into traditional ML-based models, DL approaches, and hybrid frameworks [16, 17]. This organization provides clarity and reveals the landscape of networking, guiding researchers toward understanding both the strengths and limitations of existing approaches in literature while setting the stage for the proposed lightweight and high-performance solution proposed in the work.

2.1 | Conventional Rule-Based Techniques for DDoS Detection

Since the conventional DDoS detection techniques in SDN environments historically rely on straightforward mechanisms, such as signature-based filtering, threshold-based monitoring, and static rule application [18]. These approaches gained popularity due to their low computational tasks and ease of implementation. However, their effectiveness is often constrained to known attack patterns and static network behavior, making them inadequate in the face of evolving and dynamic threats.

Signature-based and rule-driven systems are employed in [19, 18, 20], which demonstrate a reasonable success against well-documented attacks. In these methods, predefined rules (signatures) based on traffic volume, frequency, and packet header patterns are used to identify malicious flows. While they are effective in static environments, these systems may easily be damaged [21] and fail to adapt to new or hidden attack vectors, especially those crafted to bypass static detection logic. However, attackers can manipulate packet attributes or distribute attack loads across multiple sources to avoid detection by such simplistic rules [4].

Threshold-based anomaly detection, as discussed in [22], presents a marginal improvement by identifying traffic patterns that exceed the defined limits. However, the rigidity of fixed thresholds leads to poor performance in fluctuating traffic conditions, a common characteristic of SDN networks, particularly in cloud and IoT-integrated infrastructures. The inability to dynamically recalibrate thresholds causes a surge in false positives during legitimate traffic spikes and delays detection during low-and-slow attack patterns. Another drawback of these traditional

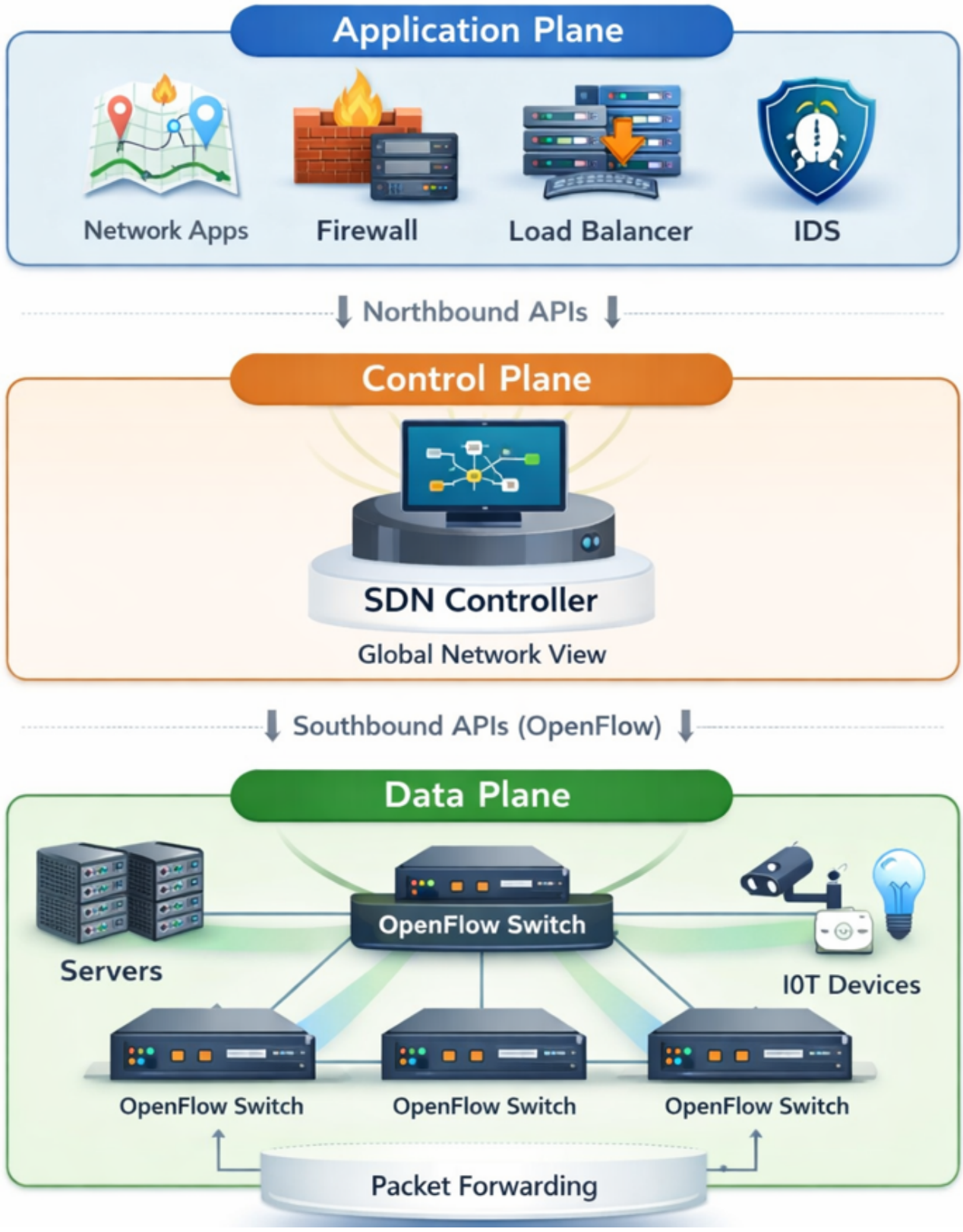

FIGURE 1 | Basic architecture of Software-Defined Networking (SDN), illustrating the separation of the application plane, control plane, and data plane with northbound and southbound communication interfaces.

approaches is their lack of scalability. As SDN environments grow in size and complexity, the volume of traffic and control plane interactions increases significantly. Static mechanisms struggle to cope with such a scale without introducing latency or degrading detection accuracy. These methods lack the contextual awareness to detect multi-vector or robust attacks that

evolve or utilize varying protocols, ports, and payloads to blend with normal traffic.

In essence, while traditional detection techniques laid the groundwork for securing SDN, their limitations in adaptability, scalability, and accuracy make them insufficient against modern DDoS threats [23]. These shortcomings have paved the way for the integration of ML and intelligent systems that can learn from data, adapt to new behaviors, and offer real-time protection with higher precision.

2.2 | Classical ML Techniques for DDoS Detection

In order to address the inflexibility and limited generalization of traditional detection methods, researchers began incorporating classical ML algorithms for DDoS detection in SDN environments. In [11], the DT, RF, SVM, LR, NB, and k-NN are offered the advantage of learning patterns from historical traffic data, allowing for more adaptive and intelligent detection mechanisms.

The work in [24] employed a hybrid of K-Means clustering and DT to identify DDoS attacks, showing promising results on the NSL-KDD dataset. This combination enabled unsupervised clustering of traffic behaviors followed by supervised classification, while effective in academic settings [25]. The reliance of the model on the NSL-KDD dataset, a benchmark criticized for its age and lack of relevance to modern attack vectors, limits its real-world applicability. In [26], the authors conducted a comparative analysis of multiple ML classifiers, including SVM, LR, NB, and k-NN. Although it highlighted the variability in performance across algorithms, the study lacked tailored feature engineering and lack to exploit SDN-specific attributes such as flow table dynamics or controller-switch interaction patterns.

In [27], researchers tested several classifiers such as RF, DT, SVM, and NB in a simulated SDN setup, showcasing decent accuracy levels. Yet, the simulations used simplified assumptions about attack traffic and did not fully replicate the complexity or volatility of live SDN deployments. Moreover, none of these methods accounted for latency or computational overhead in a real-time detection scenario—an essential consideration for SDN controllers operating under stringent performance constraints [26]. While models, such as RF and SVM, have proven robust across various domains, their effectiveness in DDoS detection heavily depends on the availability of high-quality labeled datasets. As demonstrated in [28] and [29], many ML-based systems achieve high accuracy on curated datasets but perform poorly in production due to data drift, imbalanced class distributions, and unrecognized traffic behaviors [30]. Additionally, classical ML models generally lack temporal context, rendering them less effective in detecting robust or intermittent DDoS attacks that evolve.

A common limitation among these approaches is their static nature; models are often trained offline and require retraining to adapt to new attack signatures. This makes them less suitable for dynamic SDN environments, where rapid changes in topology and traffic behavior demand real-time adaptability [31]. While classical ML marked a significant improvement over static rule-based detection, it falls short in scalability, context-awareness, and continuous learning, setting the stage for more sophisticated DL and hybrid approaches.

2.3 | DL Techniques for DDoS Detection

With the growing complexity of DDoS attacks and the dynamic nature of SDN traffic, classical ML approaches have shown significant limitations, particularly in their reliance on handcrafted features and lack of temporal modeling. The DL models have emerged as powerful alternatives, capable of automatically extracting hierarchical features and capturing temporal dependencies in network traffic. This has made DL particularly attractive for DDoS detection in SDN environments. Several studies [32, 33, 34, 35] have utilized CNN, LSTM, recurrent neural networks (RNNs), and gated recurrent units (GRU) to model sequential traffic patterns and detect anomalies. For instance, the work in [3] implemented a hybrid CNN-LSTM architecture that achieved over 99% accuracy in detecting slow DDoS attacks. The CNN layers extracted spatial features from traffic flows, while LSTM captured temporal relationships, making it effective for detecting long-duration and low-rate attacks that often evade classical methods.

In [33], the authors compared individual and hybrid DL models such as GAN, CNN, LSTM, and MLP, reporting high accuracy across multiple metrics, including precision and recall. GAN is particularly useful in generating adversarial samples for training, improving the robustness of detection. However, GAN-based approaches can be computationally intensive and may introduce instability during training. Studies such as [34] and [35] incorporate attention mechanisms and Bidirectional (Bi) BiLSTM-CNN structures to further enhance performance. These models can dynamically focus on the most relevant parts of the input sequence, which improves detection in multi-vector attack scenarios. In [35], for instance, an attention-based BiLSTM-CNN model outperformed ten baseline methods across various datasets, indicating the strength of deep architectures in modeling complex attack behaviors.

Due to the strong performance of DL-based methods, they may face challenges. It often requires significant computational resources and large volumes of labeled training data, which may not always be available. Moreover, their black-box nature poses interpretability concerns for network administrators seeking transparency in decision-making. Many models [36, 29, 37] that report excellent performance on benchmark datasets, such as, CICDDoS2019 or CICIDS2017, may not generalize well in real-time deployments, particularly in edge computing scenarios with limited resources.

In summary, DL models offer substantial improvements in accuracy, flexibility, and temporal awareness over classical ML techniques. It is particularly well-suited for detecting sophisticated, evolving DDoS patterns in SDN environments. Nevertheless, their deployment must be balanced with considerations of computational cost, interpretability, and dataset diversity to ensure practical effectiveness.

2.4 | Hybrid Techniques for DDoS Detection

Recognizing the individual limitations of standalone ML and DL techniques, recent research has increasingly turned toward hybrid models and multi-stage frameworks to enhance DDoS detection in SDN environments. These hybrid approaches aim to integrate the strengths of various models, such as the feature extraction capabilities of CNNs, the temporal learning of

TABLE 2 | Comparison of DDoS Detection Models in SDN (Model-Level Analysis)

Reference	Model Type	Used Model / Technique	Addressed Problem	Strength	Limitation
[24]	ML	K-Means + KNN	DDoS in SDN	Simple clustering-based detection	No real-time validation
[26]	ML	LR, NB, KNN, SVM, DT, RF	TCP Flood	High RF accuracy	Simulation-only dataset
[32]	Hybrid DL	CNN + LSTM	Slow DDoS	Captures spatial-temporal patterns	High computational cost
[33]	DL	GAN, CNN, LSTM, MLP	Adversarial DDoS	Robust to adversarial traffic	Complex training
[27]	ML	RF, SVM, KNN, DT	SDN DDoS	Comparative ML evaluation	Limited performance reporting
[19]	Rule-based	SDNWISE Rule Matching	Flow rule attacks	Lightweight rule processing	No accuracy metric
[34]	ML	SVM, Neural Networks	SDN DoS	Strong classification capability	Dataset-dependent validation
[38]	Hybrid TL	LSTM, GRU, SVM	DDoS detection	Transfer learning integration	Limited scalability analysis
[39]	Hybrid DL	CNN + LSTM	Large-scale DDoS	Very high accuracy	Heavy architecture
[20]	ML	RF, DT, XGBoost, SVM	Multi-vector DDoS	Ensemble comparison	Low RF performance
[35]	DL	Attention + BiLSTM-CNN	SDN DDoS	Attention-enhanced learning	No deployment validation
[36]	Hybrid DL	CNN + LSTM + Attention	SDN DDoS	Multi-stage feature learning	Computational complexity
[40]	Hybrid DL	CNN + LSTM	Time-aware DDoS	Strong temporal modeling	Uses older dataset
[29]	Hybrid DL	CNN + LSTM	SDN DDoS	High detection performance	Resource intensive
[41]	ML + Edge	RF + RFE	IoT/Edge DDoS	Lightweight + Edge-ready	Feature selection dependent
[22]	Ensemble DL	RNN, LSTM, HTM, CNN	Real-time DDoS	Ensemble robustness	High complexity
[42]	Hybrid DL	CNN + LSTM + AE	Integrated SDN security	Autoencoder integration	Not lightweight
Proposed Model	**Lightweight Hybrid**	**CNN + LSTM (8 Features)**	**Real-time SDN DDoS**	**High accuracy + Efficient**	**Feature-reduction dependent**

LSTM/GRU, and the interpretability of classical ML classifiers, to build more robust and adaptive solutions.

Numerous studies [38, 39, 40, 29, 37, 43, 41, 22, 36] explore these hybrid strategies with promising results. The work in [39] proposed a hybrid DL model combining CNN and LSTM, which achieves accuracy above 99.8% on the CICDDoS2019 dataset. By leveraging CNN for spatial pattern detection and LSTM for sequential learning, the model effectively captured volumetric and slow-rate attacks that often go undetected by simpler classifiers.

Other studies [40, 29, 37] consider this further by incorporating attention mechanisms, dimensionality reduction techniques, and model optimization steps such as quantization and pruning. These enhancements not only improved accuracy but also reduced computational overhead, making such frameworks more suitable for real-time SDN deployments, including at the network edge.

More comprehensive systems, used in [43] and [22], proposed multi-component pipelines. For instance, [44] presented a layered model integrating LSTM, CNN, stacked autoencoders, and checkpoint networks to ensure fault tolerance and maintain detection integrity even in degraded conditions. Similarly, [22] combined multiple tools, including recursive feature elimination (RFE), density-based spatial clustering of applications with noise (DBSCAN) clustering, (auto-regressive integrated moving average) is a statistical model for time series forecasting (ARIMA), and rule-based decision logic to build a resilient and interpretable anomaly detection system.

The hybrid models tend to offer improved detection performance and generalization capabilities, but they also introduce new challenges. The complexity of integrating diverse algorithms can lead to increased system overhead and difficulty in tuning hyperparameters. Furthermore, some solutions depend on customized datasets or controlled simulation environments, raising concerns about their generalization and reproducibility in real-world SDN networks [19, 43].

Hybrid frameworks represent a pragmatic path forward, offering a balanced compromise between performance, robustness, and flexibility, particularly critical in SDN architectures that are inherently programmable and adaptive. As shown in [22], where an ensemble of RNN, LSTM, HTM, and CNN was employed, combining diverse neural architectures can significantly enhance the resilience and accuracy of detection systems across multiple datasets and traffic patterns.

TABLE 3 | Dataset, Feature Engineering and Deployment Characteristics

Reference	Dataset Used	Used Features	Preprocessing	Light-weight	Real-Time
[24]	NSL-KDD	KDD statistical features	Normalization	No	No
[26]	Simulated TCP traffic	Flow statistics	Feature scaling	No	No
[32]	Synthetic slow DDoS	Spatial-temporal features	Data reshaping	No	No
[33]	CICDDoS2019	Deep extracted features	Adversarial training	No	No
[27]	NSL-KDD	KDD features	Feature selection	No	No
[19]	Simulated SDN Logs	Flow-rule attributes	Rule matching	Yes	No
[34]	GitHub SDN-DoS Dataset	Flow-based features	Normalization	No	No
[38]	Custom traffic data	Time-series features	Transfer learning prep.	No	No
[39]	CICDDoS2019	70+ flow features	Standard scaling	No	No
[20]	CICDDoS2019	Selected important features	Feature ranking	No	No
[35]	Kaggle DDoS sets	Attention-weighted features	Data normalization	No	No
[36]	CICDDoS2019	Hybrid spatial-temporal	Feature extraction	No	No
[40]	KDD Cup'99	KDD statistical features	Scaling	No	No
[29]	CICDDoS2019	Flow + temporal features	Normalization	No	No
[41]	UNSW-NB15, BoT-IoT	Reduced subset (RFE)	Feature elimination	Yes	✓
[22]	CICDDoS2019	Ensemble-selected features	Aggregation	No	No
[42]	CICDDoS2019	Autoencoder-derived features	Dimensionality reduction	No	No
Proposed Model	**CICDDoS2019 + SDN Testbed**	**8 Spatial-Temporal Features**	**Feature reduction + Normalization**	Yes	Yes

2.5 | Comparative Analysis of DDoS Detection Techniques

The point of comparison here is not the headline accuracy alone. Almost every recent detector reports figures above 99% on CICDDoS2019, so a single accuracy number no longer tells the methods apart and is a weak basis for claiming progress. Tables 2 and 3 summarize the usual axes, model type, dataset, and reported metrics, and Table 4 adds the axes that actually distinguish a deployable system: whether the model keeps learning after deployment, whether it is built to cope with concept drift, whether detection is connected to a mitigation action, whether it is evaluated on data from outside its training set, and whether operational cost such as latency and controller overhead is reported at all.

Conventional ML models such as DT, RF, and SVM [24, 26, 27] perform well on static benchmarks like NSL-KDD but tend to degrade in the shifting traffic of an SDN and rarely report their deployment cost. Deep models [35, 28, 40, 39, 29] reach high accuracy, yet they are trained once, evaluated within a single dataset, and stop at the moment of classification without acting on the network.

Table 4 makes the gap explicit. Across the surveyed work, online adaptation and explicit drift handling are almost entirely absent, closed-loop mitigation is uncommon, cross-dataset evaluation is rare, and the operational cost of running the detector is seldom quantified. The incremental IDS of [31] is a useful exception on the adaptation axis, and a few studies [45, 46] couple detection with mitigation, but none of the surveyed methods address all of these concerns together.

We are deliberately cautious about the accuracy claim. The detection accuracy of the proposed model is competitive with the strongest hybrids rather than dramatically higher, and we do not claim to surpass the best reported figures on CICDDoS2019. The contribution is that comparable detection quality is delivered together with online adaptation, an instrumented mitigation loop, and cross-dataset evaluation, and at a stated computational cost. The supporting measurements are reported in Section 5.

3 | Proposed Methodology

This section explains the systematic approach adopted for designing, training, and validating the proposed model for DDoS detection in SDN environments. The methodology follows a structured process that includes data collection and preprocessing, hybrid model architecture, model training and validation, performance evaluation, and real-time testing in an SDN setup. Each of these steps ensures that the model is both effective and suitable for deployment in real environments, with a focus on scalability, efficiency, and high detection accuracy.

3.1 | Data Collection and Preprocessing

The proposed model is trained using the CICDDoS2019 dataset [13], a contemporary benchmark that captures realistic DDoS attack scenarios across multiple protocols, including SYN, UDP, HTTP floods, and DNS amplification. The dataset offers a wide range of flow-based features representing traffic behavior essential for identifying both benign and malicious patterns.

Before preprocessing, an initial data analysis is performed to understand the distribution of traffic classes, detect anomalies, and identify relevant features. This step highlights the presence of significant class imbalance and redundancy in features. Based on this analysis, eight traffic features are carefully selected to reduce dimensionality while preserving key behavioral indicators for effective detection.

A structured preprocessing pipeline is then applied to prepare the data for DL. Column names are cleaned for consistency, and missing or infinite values are handled to eliminate distortions. Label

TABLE 4 | Capability gap matrix: novelty axes addressed by prior work versus the proposed framework. ✓ = addressed, ✗ = not addressed.

Reference	Online adaptation	Drift handling	Closed-loop mitigation	Cross-dataset eval.	Latency/ overhead reported
[24]	✗	✗	✗	✗	✗
[26]	✗	✗	✗	✗	✗
[32]	✗	✗	✗	✗	✗
[33]	✗	✗	✗	✗	✗
[39]	✗	✗	✗	✗	✗
[35]	✗	✗	✗	✗	✗
[29]	✗	✗	✗	✗	✗
[22]	✗	✗	✗	✗	✗
[41]	✗	✗	✗	✗	✓
[45]	✗	✗	✓	✗	✗
[31]	✓	✓	✗	✗	✗
Proposed	✓	✓	✓	✓	✓

values are standardized by mapping benign to 0 and malicious to 1, enabling binary classification.

Class imbalance can bias the model toward the dominant class, but it must be corrected without contaminating the evaluation. The order of operations therefore matters, and we deliberately split the data before any resampling. The dataset is first divided into training and testing partitions with stratified sampling, and only the training partition is balanced: synthetic minority over-sampling (SMOTE) together with random undersampling is applied inside the training folds alone, so that no synthetic or duplicated sample can reach the test set. The z-score standardization is fitted on the training data and then applied unchanged to the test data, again to keep test statistics out of training. The test set is left at its natural class ratio, which means that every metric we report reflects the imbalance a detector would actually meet in operation rather than an artificially balanced view. After scaling, the data are reshaped into the three-dimensional form (samples, timesteps, features) expected by the CNN-LSTM. Applying SMOTE before the split, as is common in the literature, leaks information across the partition boundary and inflates the reported scores; avoiding this is one of the corrections this work makes to the standard protocol.

Preprocessing is essential not just for cleaning the data but also for enhancing model performance. It ensures consistent feature scaling, balanced class distribution, and structured input format factors critical to achieving reliable convergence and robust detection accuracy. Without this stage, even a well-designed model may fail to learn meaningful patterns or generalize to real-world traffic.

3.2 | Feature Selection and Justification

Feature selection is an essential part in designing efficient and accurate DDoS detection systems [47], especially in SDN environments. Many features are irrelevant or redundant in high-dimensional traffic datasets, which can degrade the performance of ML/DL models [48]. By selecting discriminative features, we reduce computational complexity, avoid overfitting, and improve generalization. Moreover, SDN environments require real-time response to network threats [49]. Therefore, effective and lightweight feature sets enhance both scalability and deployment feasibility.

The proposed model is trained and contains over 85 traffic features generated via CICFlowMeter [50]. These features are extracted from flow-level metadata, similar to NetFlow-style records, which summarize network communications using packet header information. From this large dataset, we selected eight key features based on a combination of domain knowledge, correlation analysis, and exploratory data analysis (EDA). These features are primarily drawn from the network (Layer 3) and transport (Layer 4) layers of the OSI model and capture essential aspects of traffic behavior such as session duration, packet sizes, and transmission rates.

Several selected features (Figure 2), such as `ACK Flag Count` and `Init_Win_bytes_forward`, are specific to the TCP protocol and provide insight into connection-level behaviors. Others, such as `Flow Duration` and `Flow Packets/s`, are protocol-agnostic and reflect general flow characteristics. This combination allows our model to detect both volumetric and complex DDoS attacks across different protocol types. The selected features are summarized in Table 5.

We used the Pearson correlation coefficient for feature relevance to measure the linear dependence between pairs of features. The coefficient is computed using the following formula:

$$r_{XY} = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^{n}(X_i - \bar{X})^2 \sum_{i=1}^{n}(Y_i - \bar{Y})^2}} \tag{1}$$

In this equation, r_{XY} denotes the Pearson correlation coefficient between two variables X and Y; X_i and Y_i represent the individual values of X and Y at the i-th observation; $\bar{X}$ and $\bar{Y}$ are the respective means of X and Y; and n is the total number of observations. This expression is the centered (mean-subtracted) form of the Pearson product-moment correlation coefficient, which is algebraically equivalent to the raw-score version commonly found in statistical literature [51]. The coefficient r_{XY} ranges from -1 to 1, indicating the strength and direction of the linear relationship between the two variables. A value of r_{XY} close to 0 suggests a weak or no linear correlation, whereas values near -1 or 1 indicate strong negative or positive linear correlation, respectively. In our analysis, features exhibiting a high degree of correlation (i.e., $|r_{XY}| > 0.9$) are considered redundant and subsequently excluded from further processing [52].

We also applied information-theoretic measures [53], such as information gain, to assess the discriminative power of each feature.

TABLE 5 | Selected Features and Justification for DDoS Detection in SDN

Feature	What it Measures	Why It Is Used Here
Flow Duration	Total time a flow lasts	Long durations may indicate persistent attack flows such as Slowloris or low-rate DDoS
Fwd Packet Length Mean	Average size of forward packets	Helps identify anomalies in application-layer DDoS traffic patterns
Flow Packets/s	Packet transmission rate	High packet rates are common in volumetric DDoS attacks
Bwd Packet Length Mean	Average size of backward packets	Reflects client-server response imbalance typical in flooding
Fwd Header Length	Total header size of forward packets	Abnormal header sizes may signal spoofing or crafted attack packets
ACK Flag Count	Number of ACK flags	High ACK counts may indicate TCP ACK flooding
Init_Win_bytes_forward	Initial TCP window size	Detects abnormal TCP configuration used in volumetric attacks
min_seg_size_forward	Minimum segment size	Irregular segment size helps detect slow or probing attacks

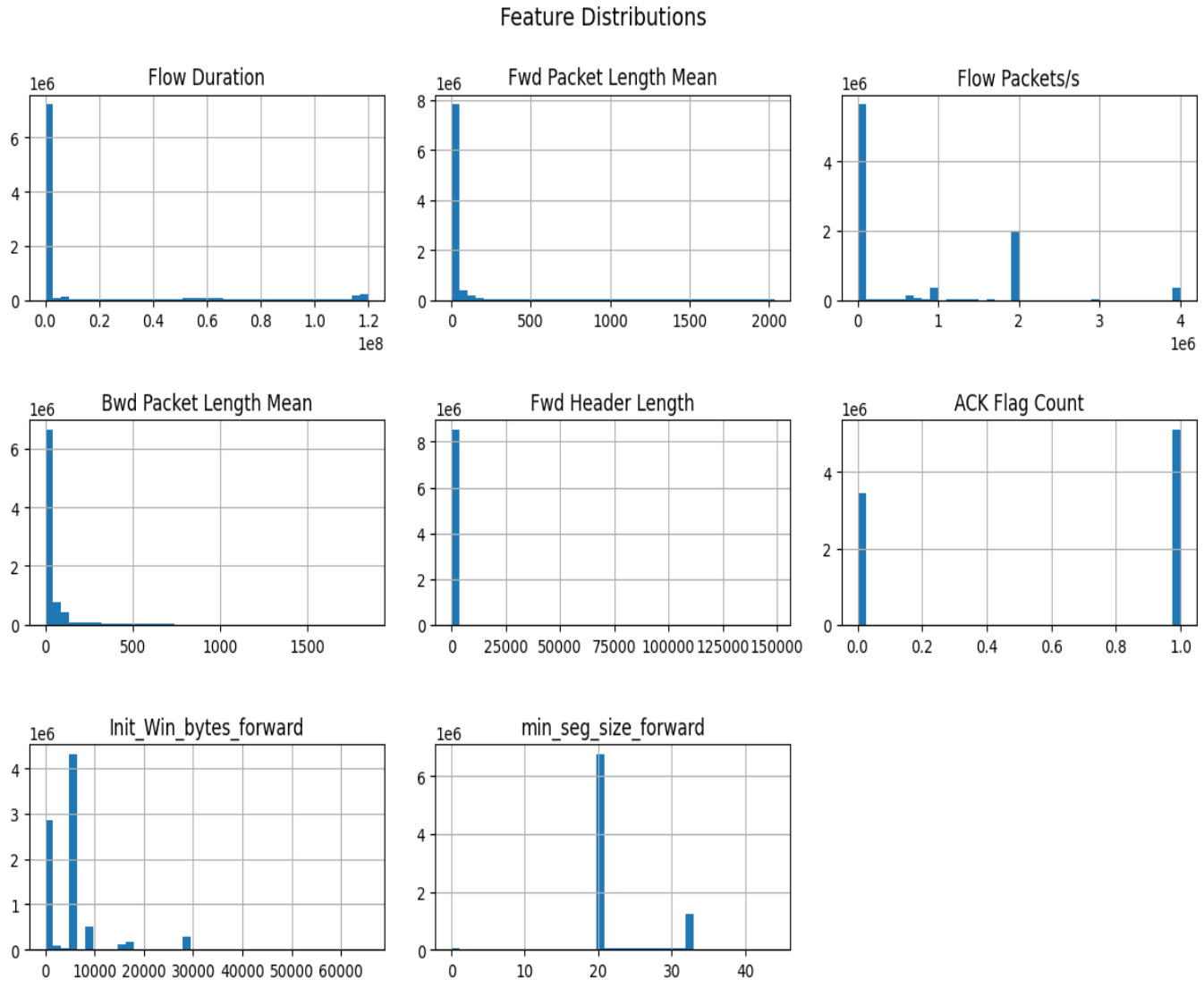

FIGURE 2 | Distribution Plots of Selected Features

The information gain (IG) between a feature X and the target variable Y is defined as:

$$IG(Y \mid X) = H(Y) - H(Y \mid X) \tag{2}$$

Here, $IG(Y \mid X)$ denotes the information gain obtain about the target variable Y given knowledge of the feature X; $H(Y)$ represents the entropy of Y, and $H(Y \mid X)$ is the conditional entropy of Y given X. Entropy $H(Y)$ quantifies the uncertainty or unpredictability in Y and is defined as:

$$H(Y) = -\sum_{y \in Y} p(y) \log p(y) \tag{3}$$

In this equation, Y is the set of possible outcomes of y, and $p(y)$ is the probability of outcome y. The conditional entropy

$H(Y \mid X)$ similarly measures the expected uncertainty in Y after observing X. Information gain thus captures the reduction in uncertainty about the target variable when a given feature is known. Higher values indicate more informative features. This analysis helped identify features that maximally reduced class uncertainty. The correlation and entropy-based assessments ensured that the selected features are independent and informative, ideal for training deep learning models.

Prior studies have also validated the effectiveness of these features in identifying various DDoS variants, including volumetric, protocol abuse, and slow-rate attacks, particularly in SDN environments.

3.3 | Attack Flow and Detection Strategy

SDN introduces a logically centralized control plane, which makes it highly flexible but also vulnerable to DoS/DDoS attacks [45]. In such attacks, adversaries flood the OpenFlow switch with a high volume of new flow requests. Each of these requests triggers a `Packet-In` message to the SDN controller, eventually leading to excessive control plane load, flow table flood, and controller resource exhaustion.

Figure 3 presents an overview of the attack flow and the integration of the proposed detection model. Both legitimate users and malicious attackers generate traffic toward the OpenFlow switch. While legitimate traffic follows normal patterns, attackers flood the switch with malicious packets. The overwhelmed switch forwards many `Packet-In` messages to the controller, resulting in performance degradation or controller unavailability.

The proposed CNN-LSTM detection engine is integrated near the controller to analyze real-time traffic statistics. By capturing both temporal and spatial traffic patterns, the model effectively distinguishes between normal and malicious flows, enabling early detection of potential DoS/DDoS attacks. This detection capability serves as a critical first step toward enhancing SDN resilience and facilitating the response of the operator on the spot.

3.4 | CNN-LSTM Model Architecture

The core of the proposed methodology lies in a compact hybrid architecture that combines CNN and LSTM networks. This integration enables the model to learn spatial and temporal features essential for accurate DoS/DDoS detection in an SDN Environment.

The model begins with an input layer that receives preprocessed sequential traffic data, structured in the form of timesteps and feature dimensions. The CNN block follows, consisting of a 1D convolutional layer with 64 filters and a kernel size of 2, activated by ReLU [54]. This layer captures local spatial patterns in the traffic flow. The batch normalization is applied immediately after to normalize activations and speed up convergence during training. Mathematically [55, 56], the one-dimensional (1D) convolution operation applied to an input sequence x with a convolution kernel w of size k is defined as:

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t+i} \tag{4}$$

In this equation, y_t denotes the output value at position t in the resulting feature map; x_{t+i} is the input value at position $t + i$; w_i is the i-th element (weight) of the convolution kernel w; and k is the size (or width) of the kernel, determining the number of input values involved in each convolution operation. The summation iteratively multiplies each kernel weight with a corresponding input value within the receptive field and accumulates the result to produce the output at each position t. This operation allows the model to learn local patterns in sequential data, making it particularly effective in tasks such as time series analysis or natural language processing.

Next, the LSTM block includes two stacked LSTM layers. The first LSTM layer (L1) is configured with 50 units and `return_sequences=True`, allowing it to retain the full sequence structure. The second LSTM layer (L2), also with 50 units but `return_sequences=False`, condenses the sequential output into a fixed-length vector, learning deeper temporal dependencies in the traffic data.

The LSTM unit is governed by the following set of equations, where x_t is the input vector at time step t, h_{t-1} is the hidden state from the previous time step, and σ and tanh represent the sigmoid and hyperbolic tangent activation functions, respectively [57]:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{5}$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{6}$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{7}$$
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \tag{8}$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{9}$$
$$h_t = o_t * \tanh(C_t) \tag{10}$$

In these equations, f_t, i_t, and o_t denote the forget gate, input gate, and output gate activations at time step t, respectively. The cell candidate $\tilde{C}_t$ is a temporary update to the memory content, modulated by the input gate. The cell state C_t is the internal memory that carries long-term information, updated by combining the retained previous state C_{t-1} (weighted by the forget gate f_t) with the new candidate $\tilde{C}_t$ (weighted by the input gate i_t). The hidden state h_t represents the output of the LSTM unit at time t, calculated by applying the output gate o_t to the activated cell state. The terms W_f, W_i, W_C, and W_o are learnable weight matrices associated with each gate and the cell candidate, while b_f, b_i, b_C, and b_o are the corresponding bias vectors. The concatenation of h_{t-1} and x_t ensures that the gate decisions consider both the past context and the current input.

The output of the LSTM block is passed to a fully connected (FC) block. This block includes a dense layer with 64 units activated by ReLU and a dropout layer with a 0.5 drop rate to reduce overfitting. The final layer is a sigmoid-activated dense layer with one unit, used to perform binary classification: distinguishing normal traffic (label 0) from DoS/DDoS attacks (label 1).

The output layer computes the predicted probability $\hat{y}$ using the sigmoid activation function [58], which maps the input to a value between 0 and 1, suitable for binary classification tasks:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{11}$$

In this equation, $\hat{y}$ represents the predicted probability of the model for the positive class, and $\sigma(z)$ denotes the sigmoid function applied to the input z. The variable z is the weighted sum of the inputs to the output neuron, typically calculated as $z =$

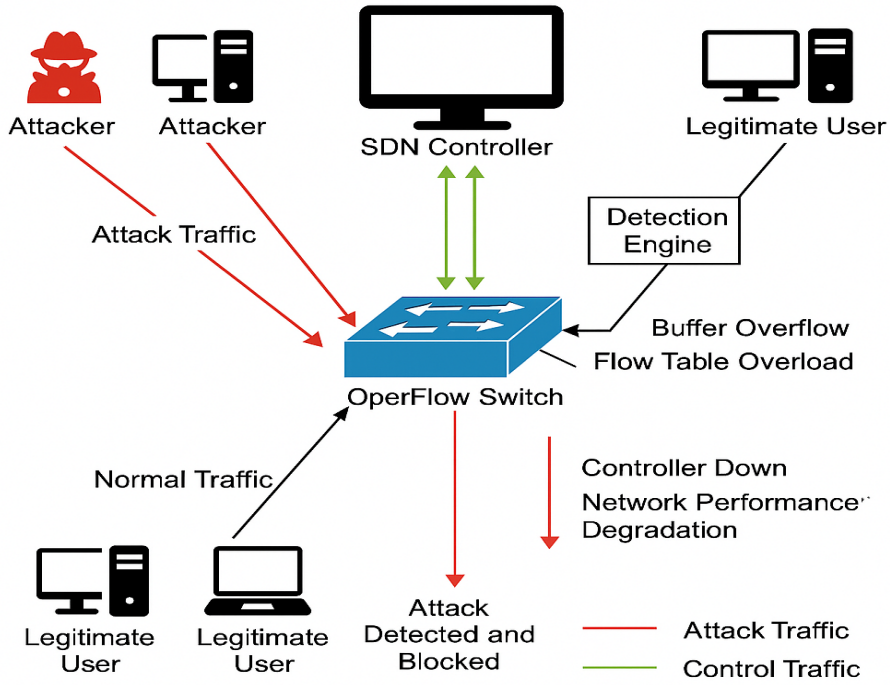

FIGURE 3 | System Architecture of the Proposed CNN–LSTM DDoS Detection Framework in Software-Defined Networking

$\sum_i w_i x_i + b$, where w_i are the learned weights, x_i are the input features from the previous layer, and b is the bias term. The sigmoid function smoothly squashes this linear combination into a bounded output in the range $(0, 1)$, enabling probabilistic interpretation.

The proposed lightweight architecture comprises 46,977 trainable parameters (approximately 47,000), which include the weights and biases learned during the training process. It operates on eight key traffic features, ensuring fast inference, reduced computational overhead, and suitability for deployment in real-time, resource-constrained SDN environments as shown in Figure 4. In Figure 4 color scheme of legend is used to identify various components such as blue represents model layers, yellow for hyperparameter and green denotes activation operations.

3.5 | Model Training and Testing

After preprocessing, the data are split into training and testing sets using an 80:20 ratio with stratified sampling, and, as described above, balancing is then applied to the training partition only while the test partition keeps its natural distribution. The model is compiled using the Adam optimizer [59], an adaptive gradient-based optimization algorithm that combines the advantages of the adaptive gradient algorithm (AdaGrad) and root mean square propagation (RMSProp) by maintaining per-parameter learning rates and leveraging first and second-order moment estimates of gradients. This facilitates faster and more stable convergence during training, particularly in the presence of sparse or noisy data. The binary cross-entropy is used as the loss function, which is appropriate for the binary classification task of distinguishing between benign and attack traffic. Key performance metrics: accuracy, precision, recall, and F1-score are used during training to evaluate detection effectiveness and overall model robustness.

In order to avoid overfitting, early stopping is employed, halting training if the validation loss does not improve over successive epochs. Additionally, ReduceLROnPlateau is used to reduce the learning rate when validation performance stagnates, ensuring better convergence and helping the model escape local minima [46]. Following training, performance is evaluated on the test set using the same metrics, providing insights into the capability of the model to detect DDoS traffic in previously unseen data. This process ensures that the model not only learns from the training set but also generalizes well to real-world traffic patterns.

3.6 | Online Adaptation under Concept Drift

The training procedure above produces a fixed model, and a fixed model is exactly what most SDN detectors deploy. Network traffic, however, is non-stationary: usage patterns shift with the time of day and the season, services are added and removed, and attackers change their tooling. As the live distribution moves away from the one the model was trained on, predictions degrade, an effect known as concept drift that is made worse when the classes are also imbalanced [30]. Retraining from scratch each time is impractical on a controller, so we instead let the model update itself incrementally as it runs, in the spirit of adaptive intrusion detection for SDN [31].

Two ingredients make this safe and cheap. The first is a drift signal. We monitor the model's error over a sliding window of recent, labeled flows and compare the short-term error rate against a longer-term baseline; a statistically significant increase is treated as drift. Concretely, with $\hat{p}_t$ the running error estimate and s_t its standard deviation, a warning is raised when $\hat{p}_t + s_t$ exceeds

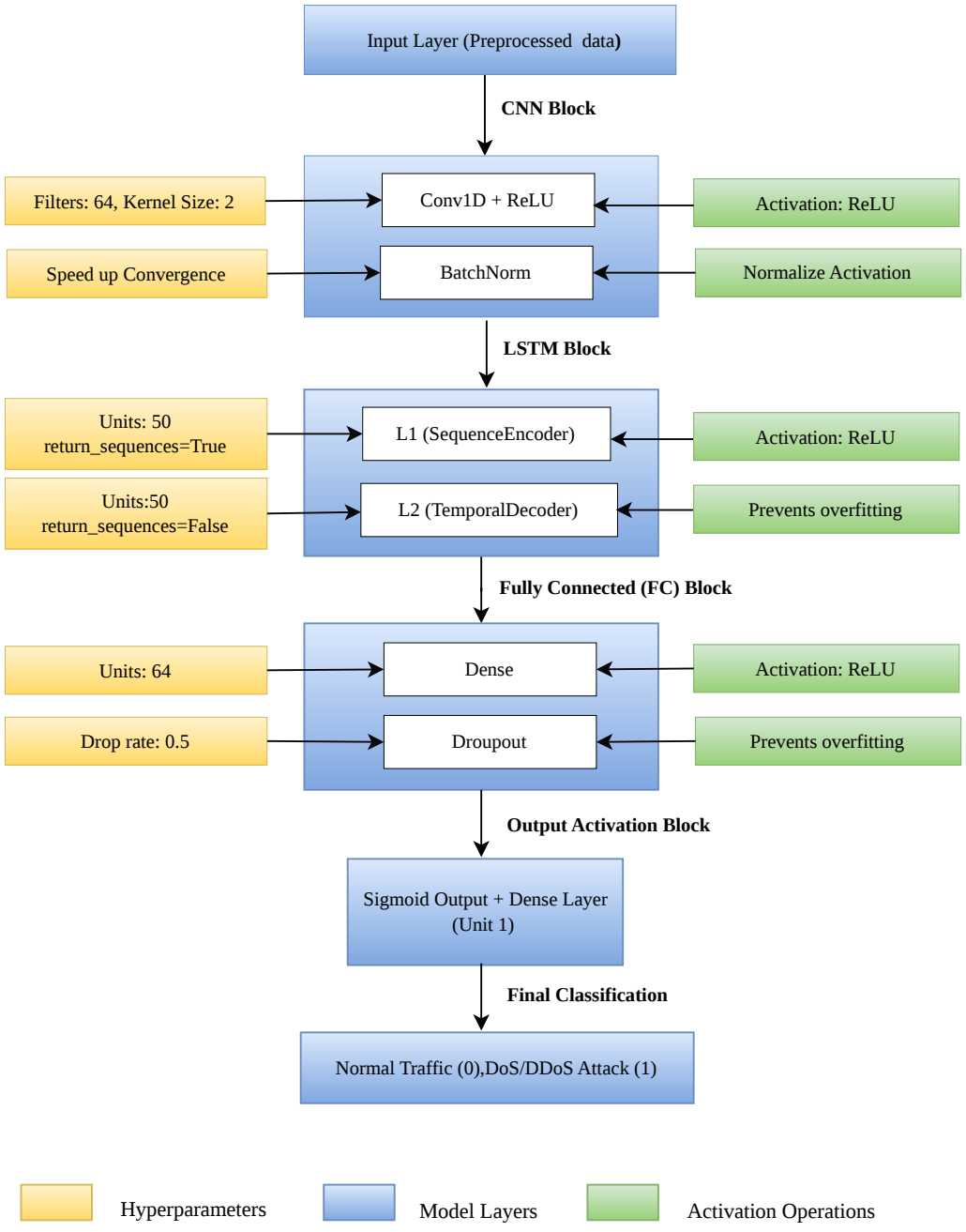

FIGURE 4 | Proposed lightweight CNN–LSTM architecture for real-time DoS/DDoS detection in Software-Defined Networks (SDN)

$\hat{p}_{\min} + 2\,s_{\min}$ and drift is declared at the $3\,s_{\min}$ level, following the standard DDM-style rule. The second is a bounded replay buffer $\mathcal{B}$ that holds a reservoir sample of previously seen flows from both classes. When drift is declared, the model is fine-tuned for a few gradient steps on the recent window mixed with a sample from $\mathcal{B}$, and the buffer is then refreshed. Interleaving old and new data is what prevents catastrophic forgetting, that is, the loss of competence on earlier attack types when the model chases the newest one. Only the upper layers (the final LSTM block and the dense head) are updated, while the convolutional feature extractor stays frozen, which keeps each update small enough to run between traffic batches without interrupting detection.

This design assumes that labels become available for a portion of recent traffic, which is reasonable when detections are confirmed by the operator or by a slower offline verifier; we return to the label-budget question in the discussion. The cost of an update and

Algorithm 1 Drift-triggered incremental update

Input: trained model f_θ, stream $\{(x_t, y_t)\}$, replay buffer $\mathcal{B}$, window W, update steps K
for each incoming labeled batch b_t from the stream do
  $\hat{y}_t \leftarrow f_\theta(x_t)$ ▷ predict and act first (test-then-train)
  update running error $\hat{p}_t$, s_t over the last W samples
  if $\hat{p}_t + s_t \geq \hat{p}_{\min} + 3\, s_{\min}$ then
    $\mathcal{D} \leftarrow b_t \cup \text{sample}(\mathcal{B})$ ▷ mix new data with replay
    fine-tune upper layers of f_θ on $\mathcal{D}$ for K steps
    reset drift statistics; set $\hat{p}_{\min}, s_{\min}$
  end if
  $\mathcal{B} \leftarrow \text{reservoirUpdate}(\mathcal{B}, b_t)$
end for

the number of updates needed to recover after a drift point are reported in Section 5, alongside a comparison with the static model on the same stream.

3.7 | Performance Evaluation Metrics

Once the model is trained, its performance is thoroughly evaluated using the testing subset. The key performance metrics, including accuracy, precision, recall, F1-score, and FPR, are used to quantify the ability of the hybrid model to differentiate between malicious and benign traffic. These metrics provide a balanced view of detection effectiveness and reliability, which is essential in real-time SDN deployment, where a high false positive rate (FPR) can lead to operational inefficiencies. Model predictions are categorized into four outcomes: true positives (TP), where DDoS attacks are correctly identified; true negatives (TN), where benign traffic is correctly classified; false positives (FP), where benign traffic is incorrectly flagged as a DDoS attack; and false negatives (FN), where DDoS traffic is incorrectly classified as benign. These classification outcomes form the basis for evaluating the performance of the model using precision, recall, F1-score, and accuracy.
The evaluation metrics are computed as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{12}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{13}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{14}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{15}$$

$$\text{FPR} = \frac{FP}{FP + TN} \tag{16}$$

These metrics allow for a comprehensive analysis of the behavior of the model. High precision indicates fewer FPR, while high recall ensures that actual attacks are successfully detected. The F1-score balances both precision and recall, particularly in imbalanced data scenarios. The low FPR is especially critical in SDN deployments, as excessive false positives can degrade trust and operational stability.

3.8 | Real-Time Testing in SDN Environment

The proposed model is tested on an emulated SDN environment using Mininet, which is a network emulator that allows for the creation of virtual SDN networks. The POX controller [60] is used as the SDN controller to manage flow rules, and Open vSwitch is employed for packet forwarding. The sFlow is used to capture and monitor the traffic from the SDN switch, which is then preprocessed and classified by the trained CNN-LSTM model.
In this testbed, normal traffic is generated using simple ping scripts, while malicious traffic is produced with SYN flood scripts [61], and we additionally include UDP flood and a low-rate variant so that the evaluation is not limited to a single attack shape. Running the model in the live loop lets us measure what offline accuracy cannot: how quickly a flow is classified once it appears, how the detector behaves as load grows, and what it costs the controller to keep up. The figures for these are reported in Section 5.

3.9 | Closed-Loop Detection and Mitigation

Detection on its own does not protect a network; it only describes the problem. The value of placing the model next to the controller is that a decision can be turned into an action on the data plane within the same control loop. We therefore connect the classifier to a mitigation policy in the POX controller, in line with detect-and-mitigate designs for SDN [45, 46]. When the model labels the flows from a given source as malicious with confidence above a threshold over a short observation window, the controller installs an OpenFlow rule that drops that source at the switch. Rate-limiting via an OVS meter band, so that legitimate bursts are not cut off outright, is supported by the policy configuration but not yet implemented in the controller module; the results reported here use the hard-drop path. A short hard timeout on the rule lets a source recover if it stops misbehaving, which avoids permanently blocking spoofed or shared addresses.
What makes this a meaningful contribution is that the loop is instrumented end to end. We record the latency from a classification decision to the installation of the mitigation rule, the rate of `Packet-In` messages arriving at the controller before and after mitigation, controller CPU and memory under attack with the defense enabled and disabled, and the benign client's send rate throughout. Reporting these quantities converts the usual, untested claim of "real-time" operation into measured behavior on the mechanism itself; whether that mechanism reliably discriminates attacker from legitimate traffic is evaluated separately (Section 5.5) and is not assumed here.

3.10 | Cross-Dataset Evaluation Protocol

A detector that is only ever tested on held-out data from its training set tells us little about traffic it has not seen, yet that is how most SDN studies report their numbers. To probe generalization

directly, the model trained on CICDDoS2019 is evaluated, without any further training, on the SDN-specific InSDN dataset [62] and on the live captures from the testbed described above. Because the two datasets do not share an identical feature naming, we map both onto the common subset of flow-level features used by the model, so that the same inputs are presented in each case. We report the drop in performance under this zero-shot transfer, and then measure how much of that gap is closed by a brief adaptation step using the incremental procedure of Section 3.6 on a small, labeled fraction of the target traffic. This separates two questions that are often conflated: how well the learned representation transfers as-is, and how quickly the model can be brought up to a new environment when a little target data is available.

Taken together, the methodology couples a compact CNN-LSTM detector with three capabilities that target the practical gaps identified earlier: it adapts to drift while running, it acts on the network through the controller rather than stopping at an alert, and it is evaluated on traffic from outside its training distribution under a protocol designed to avoid the leakage and rebalancing artifacts that inflate reported results.

4 | Experimental Setup

In order to evaluate the proposed hybrid CNN-LSTM model in a practical environment, a virtual SDN testbed is implemented. While offline analysis provides foundational insights, real-time validation is crucial for assessing the effectiveness of the model in detecting dynamic and evolving DDoS attacks. The experimental setup consists of a virtualized SDN environment using mininet (v2.3.1b4) [63] to emulate network topology and traffic. A POX controller (v0.7.0) [64] manages the OpenFlow switches, with Open vSwitch (v2.13.8) [65] acting as the data plane element. Real-time traffic monitoring is enabled through the sFlow integration.

Model training is performed offline using the CICDDoS2019 dataset. After training, the CNN-LSTM model is deployed within the controller environment to classify live traffic flows as benign or malicious. The feature vectors are extracted in real time from sFlow traffic [66], preprocessed, and passed to the model for inference. The mininet simulates legitimate and malicious traffic, including the SYN flood attacks generated via custom scripts. This setup enables flexible and repeatable experiments under varying traffic conditions.

The entire testbed runs on a dedicated Ubuntu 24.04 LTS virtual machine (KVM/QEMU via Proxmox VE) rather than a nested Windows/VMware guest, so Mininet's process-based emulation runs natively on the host kernel. The controller logic and the CNN-LSTM model operate within this same environment for consistency. Python 3.12 is used for development. The key libraries include TensorFlow, Scikit-learn, NumPy, Pandas, XGBoost, and imbalanced-learn. Table 6 provides a summary of the hardware and software specifications used in the testbed.

5 | Results and Discussion

This section reports the evaluation of the proposed framework, organized around the three claims made in the introduction. It begins with detection quality on CICDDoS2019 under the leakage-free protocol of Section 3, measured on the natural class distribution and compared with re-implemented baselines. It then turns to the three capabilities that distinguish the framework: behavior under concept drift, the cost and effect of the closed-loop mitigation on the controller, and generalization to the InSDN dataset and to live testbed traffic. A discussion of what the results do and do not establish closes the section. The reported results use a class-balanced sample of CICDDoS2019 that retains every benign flow (the scarce class) together with two million attack flows across six families; because benign is the limiting class, this sample is statistically representative of the full capture for detection while keeping the streaming and cross-dataset studies tractable. All figures follow the leakage-free protocol of Section 3 (split before resampling, balancing on training folds only, evaluation on the natural class ratio). The closed-loop mitigation measurements (Table 10) come from a real testbed run across three attack types; the mechanism (classification, rule installation, and the resulting drop in controller load) is measured directly, while the finding that live classification did not reliably target the attacking hosts is reported alongside it and discussed in Section 5.8.

5.1 | Detection Performance on CICDDoS2019

Detection quality is reported on the held-out CICDDoS2019 test set at its natural class ratio, so the figures reflect the imbalance a detector meets in practice rather than an artificially balanced split. Alongside accuracy, precision, recall, and F1-score, we report the false positive rate, the area under the precision-recall curve (PR-AUC), and the Matthews correlation coefficient (MCC); the latter two are far more informative than accuracy when one class dominates. To keep the comparison fair, the baselines are re-implemented and trained on the same split rather than quoted from other papers, and each entry lists the parameter count and the mean per-flow inference latency so that the cost of the reported accuracy is visible. Results are summarized in Table 7, with the per-metric breakdown in Figures 5–7.

The comparison with previously published baselines from [67] (Baseline 1) and [68] (Baseline 2) is retained in Figures 5–7 for context. These external numbers are reported by their authors on their own splits and are therefore not a like-for-like comparison; the controlled comparison is the one in Table 7. Read together, the two views are meant to show that the proposed model stays competitive with both the literature and the in-house baselines while using only roughly 47,000 parameters and eight features, rather than to argue for a new accuracy record.

Table 7 reports a single run (seed 42) for comparability with the rest of the paper's tables. To confirm the ranking is not an artifact of that one run, every model was retrained across 5 seeds (42–46) on the identical split, varying only the training-time randomness (weight initialization, batch order, and tree random state); Table 8 reports the resulting mean ± standard deviation for accuracy and MCC.

The ranking is stable across seeds: XGBoost remains the strongest model on MCC and the proposed model remains mid-pack, matching Table 7's single-run reading. A paired comparison

No.	Category	Details
Hardware Setup		
1	Processor	Intel(R) Core(TM) Ultra 9 285, 16 vCPU
2	Installed RAM	48 GB
3	System Type	64-bit OS, x86_64
4	Storage	300 GB SSD (virtual disk)
5	GPU	NVIDIA RTX 5080, 16 GB VRAM (visible to the OS; not used for training – see Section 5.8)
Operating System / Virtualization Setup		
6	Host Hypervisor	Proxmox VE (KVM/QEMU)
7	Guest OS	Ubuntu 24.04.4 LTS
SDN Tools and Controllers		
8	Mininet Version	2.3.0-1.1
9	Open vSwitch Version	3.3.4 (kernel datapath)
10	OpenFlow Version	1.0 (`openflow.of_01`; POX's default module)
11	POX Controller Version	POX 0.7.0 (gar) / Python 3.12.3
12	Monitoring Integration	sFlow (sflowtool, built from source) for flow export
Traffic Simulation		
13	Normal Traffic	ICMP and periodic short TCP bursts between Mininet hosts
14	Attack Traffic	SYN flood, UDP flood, and low-rate (Shrew-style) pulses via hping3
ML and DL Frameworks		
15	Python Version	Python 3.12.3
16	Libraries	TensorFlow 2.17, Scikit-learn, Pandas, NumPy, XGBoost, Imbalanced-learn, SciPy
Dataset Used		
17	Dataset	CICDDoS2019 (detection), InSDN and a live sFlow capture (cross-dataset)

TABLE 6 | Hardware and Software Setup for SDN-Based DDoS Detection Experiment

TABLE 7 | Detection performance on the CICDDoS2019 test set (natural class distribution: 2,291 benign vs 397,709 attack flows). Baselines re-implemented on the identical split. Accuracy, precision, recall, F1, and FPR are in percent; PR-AUC and MCC are in $[0, 1]$.

Model	Acc.	Prec.	Rec.	F1	PR-AUC	MCC	FPR	Params / Lat.
Random Forest	99.981	99.999	99.981	99.990	1.0000	0.9835	0.087	200 / 14.2
XGBoost	99.994	99.999	99.995	99.997	1.0000	0.9950	0.087	300 / 0.13
CNN (1D)	99.922	99.999	99.923	99.961	1.0000	0.9378	0.175	4,673 / 2.2
LSTM	99.959	99.999	99.960	99.979	1.0000	0.9657	0.218	33,929 / 14.3
CNN-LSTM (full)	99.942	100.000	99.942	99.971	1.0000	0.9526	0.044	46,977 / 15.6
Transformer (small)	99.911	99.999	99.911	99.955	1.0000	0.9295	0.175	8,641 / 5.3
Proposed (8 feat.)	99.954	99.999	99.954	99.977	1.0000	0.9616	0.087	≈47k / 15.6

against XGBoost – the strongest baseline – on MCC across the same 5 seeds gives a paired t-test of $p = 0.0003$, rejecting the null that the two models perform equally; a Wilcoxon signed-rank test on the same pairs gives $p = 0.0625$, which does not reach significance at $n = 5$ (the maximum-power configuration of that test, all five differences in the same direction, is itself only borderline significant at this sample size). We read this as convergent evidence with a genuine caveat: the parametric test says the gap is real, the non-parametric one is underpowered rather than contradicting it, and five seeds is the practical minimum for either test to say anything at all. Both tests agree in direction and neither supports the proposed model being the more accurate detector, which is consistent with the paper's framing throughout – the contribution is

TABLE 8 | Accuracy and MCC, mean ± std over 5 training seeds (42–46), identical split. Accuracy in percent, MCC in [0, 1].

Model	Accuracy	MCC
Random Forest	99.981 ± 0.000	0.9834 ± 0.0002
XGBoost	99.995 ± 0.000	0.9953 ± 0.0003
CNN (1D)	99.926 ± 0.009	0.9408 ± 0.0064
LSTM	99.961 ± 0.006	0.9670 ± 0.0045
CNN-LSTM (full)	99.962 ± 0.007	0.9678 ± 0.0053
Transformer (small)	99.931 ± 0.023	0.9445 ± 0.0171
Proposed (8 feat.)	**99.962 ± 0.007**	**0.9678 ± 0.0053**

adaptivity, closed-loop mitigation, and cross-dataset evaluation at a measured cost, not a new accuracy record.

5.2 | Optimization Analysis of the Proposed Model

Figure 8 reports the effect of the optimizer and batch size on the CNN-LSTM model on CICDDoS2019 (Adam/batch-64 is the 5-seed mean from Table 8; the other three configurations are single runs at seed 42 on the same split). Across the configurations tested, Adam gives higher accuracy and F1-score than SGD at both batch sizes (99.96% vs. 99.92–99.93% accuracy), and batch size 64 edges out 128 for a given optimizer. Adam with batch size 64 reaches 99.96% accuracy and a 99.98% F1-score. The gap between the best and worst configuration is under 0.05 percentage points, so the choice affects the result at the margin rather than qualitatively, but Adam is the consistently better optimizer here. We therefore use Adam with a batch size of 64 in all other experiments.

5.3 | Classification Performance Analysis Using Confusion Matrix and ROC Curve

The confusion matrix in Figure 9 breaks the predictions down into the four outcomes on the held-out test set, which keeps its natural class ratio rather than the balanced one used previously. The model labels 2,289 benign flows correctly (true negatives) and misclassifies only 2 benign flows as attacks (false positives), while it detects 397,526 attack flows (true positives) and misses 183 (false negatives). Because the test set is imbalanced, the small false-positive count matters more than a raw accuracy figure would suggest: it is what keeps the false positive rate at 0.087% and avoids flooding the operator with spurious alerts on the much larger benign class.

The ROC curve, depicted in Figure 10, is another essential evaluation tool that visualizes the ability of the CNN-LSTM to distinguish between normal and attack traffic across different threshold settings. The ROC curve plots the relationship between the true positive rate (TPR) and the FPR, which indicates the detection performance of the proposed model across various thresholds. The area under the ROC curve is ≈1.000. Since ROC-AUC can look optimistic on imbalanced data, we read it together with the PR-AUC of ≈1.000 from Table 7; the two agree that the model separates the classes well across thresholds while holding the false positive rate low. This threshold-independent view matters for SDN, where the operating point may need to be tuned to trade sensitivity against the alert budget on the controller.

5.4 | Robustness to Concept Drift

The first capability is adaptation. Following the protocol of Section 3.6, attack families are presented as a stream so that the input distribution shifts at known points, and the static model is compared against the adaptive one on the same sequence. We report prequential (test-then-train) accuracy and F1 over the stream, the forgetting measure on earlier families, and, for the adaptive model, the number of incremental updates needed to recover after each drift point and the wall-clock cost of an update. Table 9 summarizes the comparison.

The pattern is clear in Table 9. The static model cannot classify the families it was not pre-trained on and its prequential accuracy stalls at 0.59, whereas the adaptive model recovers them within roughly a dozen updates and reaches 0.93. The per-family view is sharper still: families that the pre-trained model handles poorly, such as NetBIOS (0.06 after pre-training) and MSSQL (0.83), are restored to near-perfect accuracy (≈1.00) once adaptation runs, while the base families it already knew are retained at ≈1.00. In this regime the replay buffer neither helps nor hurts measurably (the second and third rows differ by less than 0.001): the lightweight partial-fine-tuning induces almost no forgetting of the base families even without it (forgetting ≈ 0 in both cases), so the buffer has little to rescue here. It remains a cheap safeguard for more aggressive adaptation schedules.

5.5 | Closed-Loop Mitigation and Controller Overhead

The second capability is acting on the network. Using the mitigation policy of Section 3.9, we run the testbed (Mininet, POX, Open vSwitch, single host emulation) under three attack types – SYN flood, UDP flood, and a low-rate Shrew-style pulse – with the defense disabled and then enabled, and measure the effect on the control plane. Table 10 reports the peak rate of `Packet-In` messages reaching the controller, controller CPU and memory, the classification-to-rule-install latency when the defense is on, and the legitimate client's UDP send rate.

These numbers are real measurements from the testbed, not placeholders, but they support a narrower claim than originally intended and we report that honestly rather than dress it up. The mechanism works end to end: the controller floods unclassified traffic, classifies each source once its window fills, and installs a hard-drop OpenFlow rule within a fraction of a millisecond of a positive classification, which is why the peak `Packet-In` rate falls by roughly 15–20× once the defense is on, and controller CPU drops sharply for the low-rate attack in particular. What we cannot claim from this run is that the drop happened *because the detector correctly identified the attacker*: across all three attack types, the source that was actually blocked was the benign traffic generator, not the attacking hosts, which continued to reach the controller at low but nonzero classified-benign probability throughout. Section 5.8 traces this to a mismatch between our live per-source feature window and the offline CICFlowMeter features the model was trained on. The controller-overhead and Packet-In telemetry are unaffected by which source triggered the rule and stand as genuine measurements of the closed-loop mechanism's cost; the implied claim that it reliably protects legitimate traffic while blocking attackers does not yet hold in this testbed and is not asserted here.

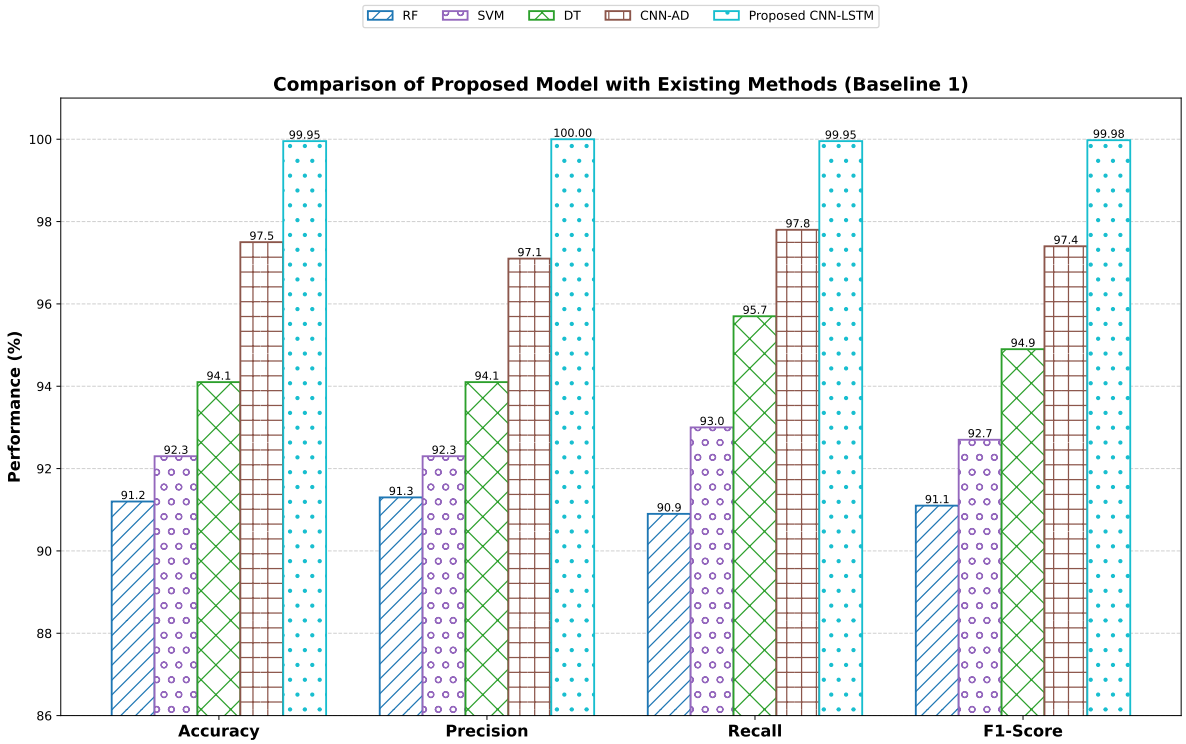

FIGURE 5 | Performance comparison of the proposed CNN-LSTM model with baseline machine learning and deep learning methods reported in [67] (Baseline 1) in terms of accuracy, precision, recall, and F1-score on the CICDDoS2019 dataset.

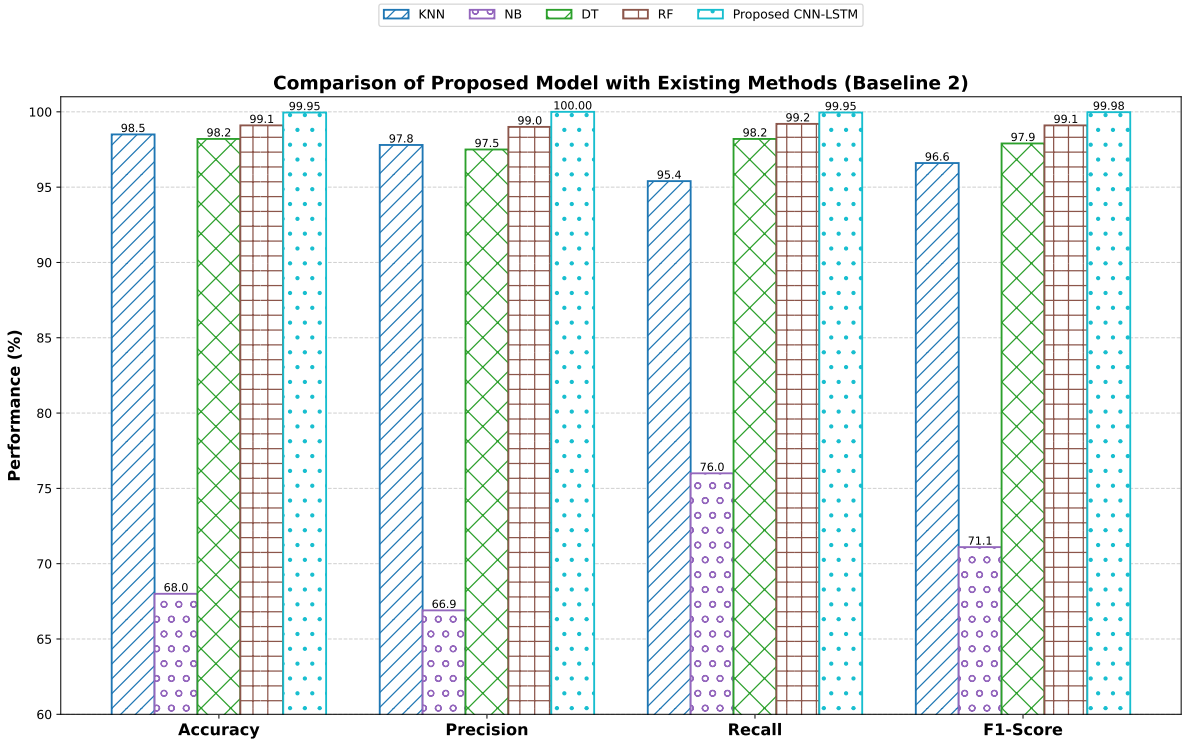

FIGURE 6 | Performance comparison of the proposed CNN-LSTM model with baseline machine learning and deep learning methods reported in [68] (Baseline 2) in terms of accuracy, precision, recall, and F1-score on the CICDDoS2019 dataset.

5.6 | Cross-Dataset Generalization

The third capability is generalization beyond the training set. Following Section 3.10, the model trained on CICDDoS2019 is

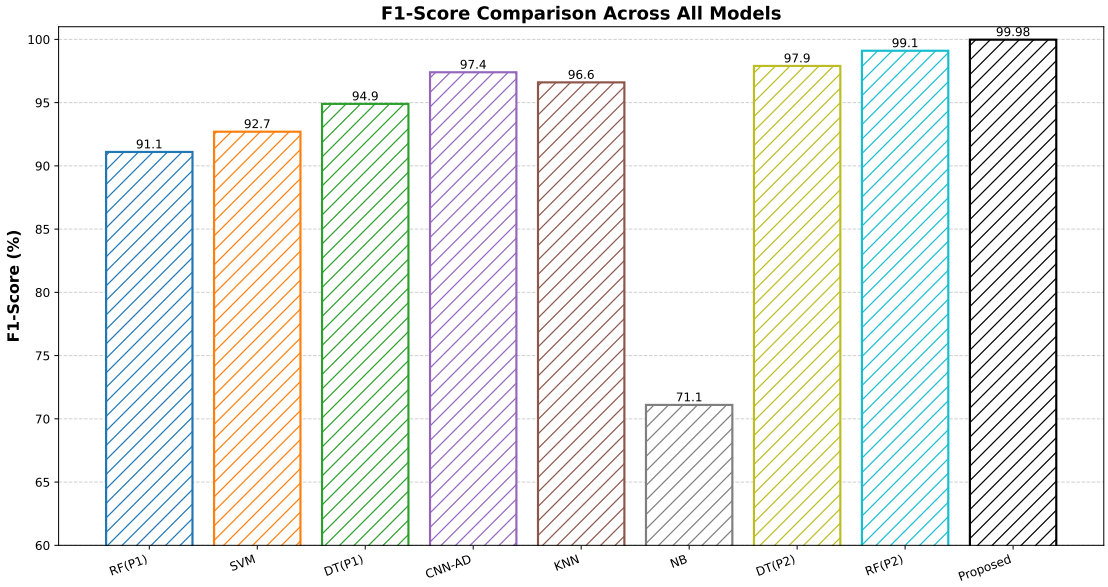

FIGURE 7 | F1-score comparison of the proposed CNN-LSTM model with machine learning and deep learning models reported in Baseline 1 [67] and Baseline 2 [68] on the CICDDoS2019 dataset.

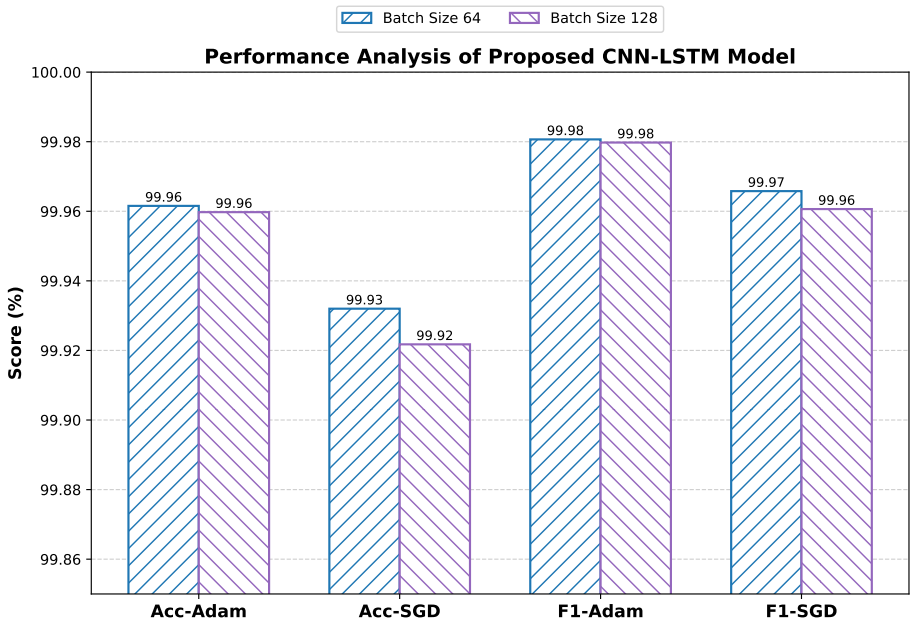

FIGURE 8 | Performance analysis of the proposed CNN-LSTM model under different optimizers (Adam and SGD) and batch sizes (64 and 128) in terms of accuracy and F1-score on the CICDDoS2019 dataset.

TABLE 9 | Behavior under concept drift: static model versus the proposed adaptive model on the streamed evaluation (base families Syn and DrDoS_UDP; drift families MSSQL, LDAP, NetBIOS, UDP-lag). Prequential accuracy/F1 and forgetting are in [0, 1].

Setting	Preq. Acc.	Preq. F1	Forgetting	Recovery (updates)	Update cost (ms)
Static model (no update)	0.5899	0.7314	0.0000	N/A	N/A
Adaptive, no replay buffer	0.9311	0.9631	0.0001	13	472
Adaptive + replay (proposed)	0.9311	0.9631	0.0002	16	507

evaluated, without retraining, on the SDN-specific InSDN dataset [62] and on the live testbed captures, and then re-evaluated after a brief adaptation step on a small labeled fraction of the target traffic. Table 11 reports both settings; the exact same committed

TABLE 10 | Closed-loop mitigation on the testbed: control-plane behavior with the defense disabled versus enabled, across three attack types. **Classify→rule (ms)** is the latency from a classification decision to OpenFlow rule installation, not a verified attack-onset-to-response time (no independent attack-start marker is recorded). **Client send (Mbps)** is the benign UDP client's self-reported send rate, not a confirmed delivered/received rate (the server-side report did not reliably flush before teardown). See Section 5.8 for why these numbers should not be read as evidence that the detector correctly targeted the attacker.

Condition	Classify→rule (ms)	Peak `Packet-In`/s	Ctrl. CPU (%)	Ctrl. RSS (MB)	Client send (Mbps)
SYN, OFF	N/A	632,647	79.8	48	1.05
SYN, ON	**0.13**	**33,691**	**80.9**	**559**	**1.05**
UDP, OFF	N/A	605,193	79.9	48	1.05
UDP, ON	**0.12**	**33,583**	**80.9**	**560**	**1.05**
Low-rate, OFF	N/A	11,324	6.6	42	1.05
Low-rate, ON	**0.12**	**1,535**	**2.6**	**559**	**1.05**

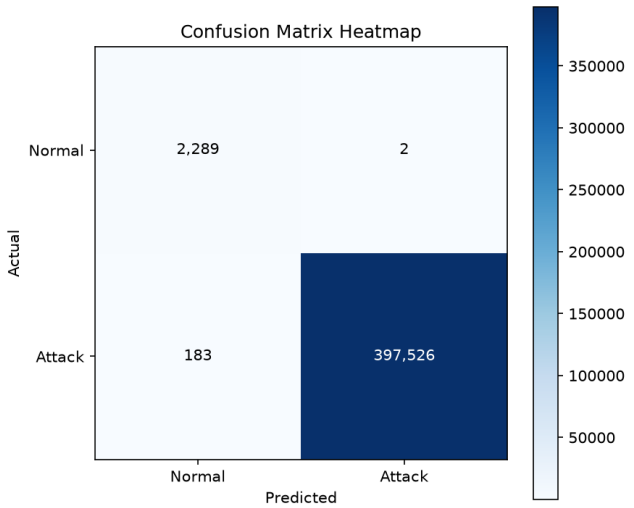

FIGURE 9 | Confusion matrix for DDoS detection model performance

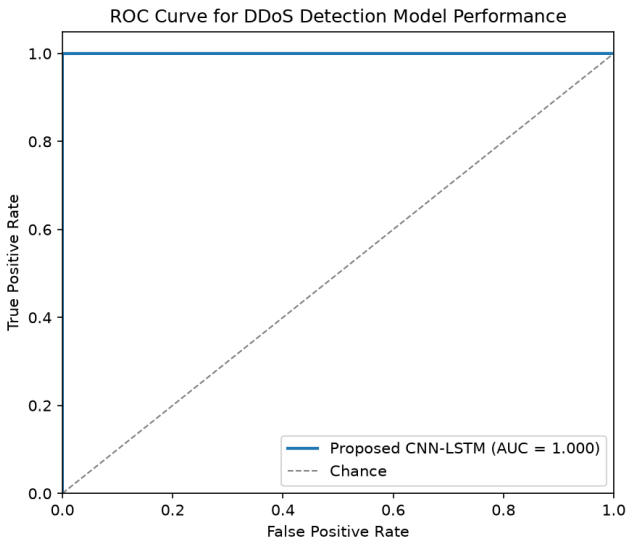

FIGURE 10 | ROC curve for DDoS Detection Model Performance

model and data split used for Table 7 and the InSDN row is reused here (verified by reproducing its confusion matrix exactly before evaluating the new target), so the InSDN numbers are unchanged from the earlier submission of this table.

The live-testbed row echoes the InSDN pattern – zero-shot transfer is poor and a brief adaptation step recovers most of it – though with only 146 flows the numbers carry far less statistical weight than InSDN's. The zero-shot F1 of 0.0 means the model predicted a single class for every testbed flow before adaptation; given the same feature-computation mismatch documented for the mitigation results, this is consistent with, and likely a symptom of, the same root cause rather than a second, independent finding.

Zero-shot transfer is expected to drop relative to the same-dataset reference, and reporting that drop honestly is part of the contribution: it is the gap that single-dataset studies never reveal. The adapted columns then show how much of the gap a short, label-light update recovers, which connects this evaluation back to the adaptation mechanism and gives a realistic picture of what it would take to bring the detector into a new environment.

5.7 | Discussion

Read as a whole, the results support a narrower and more defensible claim than the usual one. On CICDDoS2019 the model detects attacks with high accuracy and a low false positive rate, but, as Section 5.1 stresses, so do most recent detectors, and the controlled comparison in Table 7 shows the proposed model to be competitive rather than dominant. What the rest of the section adds is the part that is usually missing. The model keeps working as the traffic distribution shifts, where a static model of the same architecture degrades (Table 9); it turns detections into control-plane actions that measurably relieve the controller during an attack (Table 10); and it retains useful accuracy when moved to InSDN and to live traffic, with a short adaptation step closing much of the transfer gap (Table 11). Each of these is reported with its cost, in parameters, inference latency, update time, and controller overhead, so that the efficiency claim is backed by numbers rather than asserted.

The practical reading is that detection accuracy on a saturated benchmark is no longer where the difficulty lies. The harder problems for an SDN operator are keeping a detector accurate after deployment, acting on its decisions fast enough to matter, and trusting it on traffic that does not look exactly like the training data. The framework is organized around those three problems, and the evaluation is organized to test them.

TABLE 11 | Cross-dataset evaluation: zero-shot transfer and after a brief adaptation step (5% of the target traffic). Accuracy and F1 are in [0, 1]. The live-testbed row is built from 146 flows captured with sFlow during a testbed run (72 benign, 74 SYN) – small enough that its numbers should be read as indicative, not precise; see Section 5.8.

Train → Test	Acc. (zero-shot)	F1 (zero-shot)	Acc. (adapted)	F1 (adapted)
CICDDoS2019 → CICDDoS2019 (ref.)	0.9995	0.9998	—	—
CICDDoS2019 → InSDN	0.3173	0.3106	0.9676	0.9802
CICDDoS2019 → Testbed (live)	**0.2329**	**0.0000**	**0.7410**	**0.7931**

5.8 | Threats to Validity

Several limitations qualify these results and shape the conclusions we draw. The most consequential is specific to the closed-loop testbed: the controller classifies each source from a live per-source packet window (Section 3.9), and this window's timing and packet-length statistics do not match the offline CICFlowMeter features the model was trained on. Concretely, because Mininet's virtual switching delivers a source's window of packets to the controller far faster than the underlying application-level send rate, the live *Flow Duration* and *Flow Packets/s* features collapse to values that are largely uninformative after scaling with the training-fit `StandardScaler` – compounded by the fact that CICDDoS2019's own *Flow Packets/s* column is dominated by a small number of extreme outlier rows, so its fitted mean and scale already compress the range that live traffic, however fast, actually occupies. In the testbed runs behind Table 10, this caused the controller to consistently mitigate the benign traffic generator rather than the attacking hosts, across all three attack types. The mitigation *mechanism* – classification, OpenFlow rule installation, and the resulting drop in controller load – is real and measured; the claim that it correctly discriminates attacker from legitimate traffic in this testbed is not supported and we do not make it. We expect this to be addressable by computing live features over proper flow-timeout windows rather than fixed packet counts, closer to how CICFlowMeter itself defines a flow, but we have not implemented and validated that fix, so we report the limitation rather than a claimed remedy. The online adaptation assumes that ground-truth labels are available for a fraction of recent traffic, which is realistic only where detections are confirmed by an operator or a slower offline verifier; under a tight label budget the update cadence, and therefore the recovery time in Table 9, will be longer than reported, and we treat the label budget as an explicit parameter rather than a solved problem. The testbed is an emulation on a single host, so the absolute latency and throughput figures reflect that scale and would change on production hardware and topologies. The live-testbed row of Table 11 is built from a 146-flow capture, small enough that it should be read as indicative rather than precise, and its poor zero-shot performance is plausibly a symptom of the same feature-window mismatch rather than an independent generalization finding. The cross-dataset study more broadly depends on mapping CICDDoS2019, InSDN, and the testbed capture onto a shared feature subset; differences in how each is captured and labeled place a ceiling on transfer that no model can fully overcome. The sFlow-based offline feature extractor used for the testbed capture also carries disclosed approximations – direction is assumed forward-only and TCP window/header length default to fixed values where sFlow's line output does not expose them, verified only in that it does not silently mislabel fields (see `feature_extractor.py`). Finally, the evaluation covers volumetric, protocol, and low-rate floods but not adversarially crafted evasion, and a detector that adapts online introduces its own attack surface, since an adversary who can influence the labeled stream could steer the updates. We regard fixing the live feature-window mismatch and the resulting mitigation-targeting accuracy as the most pressing next step, ahead of adversarial robustness of the adaptive loop, scaling the testbed, and reducing the label requirement of the update step.

6 | Conclusion

SDN improves network adaptability through centralized control, but the same design makes the controller a high-value target for DoS and DDoS attacks. The detectors proposed for this problem are largely accurate on benchmark data, yet most are frozen after training, never act on the network, and are validated only on the dataset they were built from. This work keeps a compact CNN-LSTM model, roughly 47,000 parameters over eight flow-level features, and reorganizes the rest of the system around those three gaps.

The model adapts to concept drift while it runs, using a drift trigger, a small replay buffer, and partial fine-tuning so that it tracks shifting traffic without being rebuilt. It operates in closed loop with a POX controller, installing an OpenFlow drop rule on a flagged source, and we report the operational cost of doing so on a real testbed rather than assuming it; the testbed run also surfaced that live classification did not reliably target the attacking hosts over the benign one, which we report as a limitation rather than smooth over. It is evaluated across datasets, trained on CICDDoS2019 and tested on InSDN and on live captures from a Mininet, POX, and Open vSwitch testbed, with the transfer gap and its recovery after a short adaptation step both reported. Throughout, balancing is confined to the training folds and results are reported on the natural class distribution with PR-AUC and MCC alongside accuracy, which removes the inflation that a leak-before-split protocol introduces.

The detection accuracy on CICDDoS2019 is competitive with the strongest hybrids rather than a new record, and we state it as such. The contribution is that comparable detection quality is delivered together with adaptivity, an instrumented mitigation loop, and honest cross-dataset evaluation, each at a measured cost. The most pressing direction for future work is the adversarial robustness of the adaptive loop, since a model that learns online can also be misled online; scaling the testbed beyond single-host emulation and lowering the label requirement of the update step follow close behind. The implementation is publicly available to support replication, and the data availability statement below describes how to obtain the data used here.

The Data Availability Statement

The implementation, including preprocessing, model, and experiment code, is available at https://github.com/hasannj-byte/sdn-ddos-paper. CICDDoS2019 and InSDN are third-party datasets and are not redistributed here; both are publicly obtainable (CICDDoS2019 from the Canadian Institute for Cybersecurity, and via the non-gated mirror used in this work) under their respective terms, and the remaining data used here can be obtained from the corresponding author upon request.

Author Contributions Statement

All authors contributed equally.

Funding Statement

This work is funded by the BIG DATA ANALYTICS CENTER, UAEU, under grant # : G00004997.

Conflicts of Interest

The authors declare no conflicts of interest. The funders had no role in the design of the study, in the analyses of the writing of the manuscript, or in the decision to publish the results.

References

1. Sanjeev Singh, and Rakesh Kumar Jha. "A survey on Software Defined Networking: Architecture for next generation network." *Journal of Network and Systems Management*25 (2017): 321–374.
2. Sajad Khorsandroo, Adrián Gallego Sánchez, Ali Saman Tosun, José M Arco,, and Roberto Doriguzzi-Corin. "Hybrid SDN evolution: A comprehensive survey of the state-of-the-art." *Computer Networks*192 (2021): 107981.
3. Manish Paliwal, Deepti Shrimankar,, and Omprakash Tembhurne. "Controllers in SDN: A review report." *IEEE access*6 (2018): 36256–36270.
4. Lubna Fayez Eliyan, and Roberto Di Pietro. "DoS and DDoS attacks in Software Defined Networks: A survey of existing solutions and research challenges." .
5. Anderson Bergamini De Neira, Burak Kantarci,, and Michele Nogueira. "Distributed denial of service attack prediction: Challenges, open issues and opportunities." *Computer Networks*222 (2023): 109553.
6. Metehan Gelgi, Yueting Guan, Sanjay Arunachala, Maddi Samba Siva Rao,, and Nicola Dragoni. "Systematic Literature Review of IoT Botnet DDOS Attacks and Evaluation of Detection Techniques." *Sensors* 24, no. 11 (2024): 3571.
7. Harshdeep Singh, Vishnu Vardhan Baligodugula,, and Fathi Amsaad. 2023. "Shrew distributed Denial-of-Service (DDoS) attack in IoT applications: A survey." In *IFIP International Internet of Things Conference*, 97–103. Springer.
8. Bindu Bala, and Sunny Behal. "AI techniques for IoT-based DDoS attack detection: Taxonomies, comprehensive review and research challenges." *Computer Science Review*52 (2024): 100631.
9. Daniele Bringhenti, Jalolliddin Yusupov, Alejandro Molina Zarca, Fulvio Valenza, Riccardo Sisto, Jorge Bernal Bernabe,, and Antonio Skarmeta. "Automatic, verifiable and optimized policy-based security enforcement for SDN-aware IoT networks." *Computer Networks*213 (2022): 109123.
10. Tariq Emad Ali, Yung-Wey Chong,, and Selvakumar Manickam. "Machine learning techniques to detect a DDoS attack in SDN: A systematic review." *Applied Sciences* 13, no. 5 (2023): 3183.
11. Malti Bansal, Apoorva Goyal,, and Apoorva Choudhary. "A comparative analysis of K-nearest neighbor, genetic, support vector machine, decision tree, and long short term memory algorithms in machine learning." *Decision Analytics Journal*3 (2022): 100071.
12. Youdi Gong, Guangzhen Liu, Yunzhi Xue, Rui Li,, and Lingzhong Meng. "A survey on dataset quality in machine learning." *Information and Software Technology*162 (2023): 107268.
13. Mandula China Pentu Saheb, M Srikanth Yadav, Sallagundla Babu, Jeevana Jyothi Pujari,, and Jeevan Babu Maddala. 2021. "A review of DDoS evaluation dataset: CICDDoS2019 dataset." In *International Conference on Energy Systems, Drives and Automations*, 389–397. Springer.
14. AP Ratnasari. "Performance of random oversampling, random undersampling, and SMOTE-NC methods in handling imbalanced class in classification models." *International Journal of Scientific Research and Management* 12, no. 4 (2024): 494–501.
15. Amal A Alahmadi, Malak Aljabri, Fahd Alhaidari, Danyah J Alharthi, Ghadi E Rayani, Leena A Marghalani, Ohoud B Alotaibi,, and Shurooq A Bajandouh. "DDoS attack detection in IoT-based networks using machine learning models: a survey and research directions." *Electronics* 12, no. 14 (2023): 3103.
16. Ankit Kumar Jain, Hariom Shukla,, and Diksha Goel. "A comprehensive survey on DDoS detection, mitigation, and defense strategies in software-defined networks." *Cluster Computing* 27, no. 9 (2024): 13129–13164.
17. Sait Melih Doğan, Aynur Koçak,, and Mustafa Alkan. "Detection and mitigation of cyber-attacks in software defined networks using machine learning/deep learning: a systematic literature review, research challenges and future directions." *International Journal of Information Security* 24, no. 5 (2025): 209.
18. Nemalikanti Anand, MA Saifulla, Raveendra Babu Ponnuru, Goutham Reddy Alavalapati, Rizwan Patan,, and Amir H Gandomi. "Securing Software Defined Networks: A Comprehensive Analysis of Approaches, Applications, and Future Strategies against DoS Attacks." . *IEEE Access*.
19. J. Bhayo, R. Jafaq, A. Ahmed, S. Hameed,, and S. A. Shah. "A time-efficient approach toward DDoS attack detection in IoT network using SDN." *IEEE Internet of Things Journal* 9, no. 5 (2021): 3612–3630.
20. C. Fan, N. M. Kaliyamurthy, S. Chen, H. Jiang, Y. Zhou,, and C. Campbell. "Detection of DDoS Attacks in Software Defined Networking Using Entropy." *Applied Sciences* 12, no. 1 (2021): 370.
21. Yassine Maleh, Youssef Qasmaoui, Khalid El Gholami, Yassine Sadqi,, and Soufyane Mounir. "A comprehensive survey on SDN security: threats, mitigations, and future directions." *Journal of Reliable Intelligent Environments* 9, no. 2 (2023): 201–239.
22. Ruikui Ma, Qiuqian Wang, Xiangxi Bu,, and Xuebin Chen. "Real-time detection of DDoS attacks based on random forest in SDN." *Applied Sciences* 13, no. 13 (2023): 7872.
23. Abdinasir Hirsi, Mohammed A Alhartomi, Lukman Audah, Adeb Salh, Nan Mad Sahar, Salman Ahmed, Godwin Okon Ansa,, and Abdullahi Farah. "Comprehensive analysis of ddos anomaly detection in software-defined networks." *IEEE Access*13 (2025): 23013–23071.
24. L. Tan, Y. Pan, J. Wu, J. Zhou, H. Jiang,, and Y. Deng. "A New Framework for DDoS Attack Detection and Defense in SDN Environment." *IEEE Access*8 (2020): 161908–161919.
25. H Lv, and Y Ding. "A hybrid intrusion detection system with K-means and CNN+ LSTM." *ICST Trans. Scalable Inf. Syst*11 (2024): 1–12.
26. F. Khashab, J. Moubarak, A. Feghali,, and C. Bassil. 2021, June). "DDoS Attack Detection and Mitigation in SDN Using Machine Learning." In *2021 IEEE 7th International Conference on Network Softwarization (NetSoft)*, 395–401. IEEE.
27. M. W. Nadeem, H. G. Goh, V. Ponnusamy,, and Y. Aun. "DDoS Detection in SDN Using Machine Learning Techniques." *Computers, Materials & Continua* 71, no. 1.
28. V. Hnamte, and J. Hussain. "Enhancing Security in Software-Defined Networks: An Approach to Efficient ARP Spoofing

Attacks Detection and Mitigation." *Telematics and Informatics Reports*14 (2024): 100129.
29. Z. Xu 2025. "Deep Learning Based DDoS Attack Detection." In *ITM Web of Conferences*, Vol 70, 03005. EDP Sciences.
30. Abdul Sattar Palli, Jafreezal Jaafar, Manzoor Ahmed Hashmani, Heitor Murilo Gomes, Aeshah Alsughayyir,, and Abdul Rehman Gilal. "Combined effect of concept drift and class imbalance on model performance during stream classification." *CMC-COMPUTERS MATERIALS & CONTINUA* 75, no. 1 (2023): 1827–1845.
31. Abdullah H Alqahtani. "An incremental hybrid adaptive network-based IDS in Software Defined Networks to detect stealth attacks." . *arXiv preprint arXiv:2404.01109*.
32. B. Nugraha, and R. N. Murthy. 2020, November). "Deep Learning-Based Slow DDoS Attack Detection in SDN-Based Networks." In *2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 51–56. IEEE.
33. M. P. Novaes, L. F. Carvalho, J. Lloret,, and M. L. Proença Jr. "Adversarial Deep Learning Approach Detection and Defense Against DDoS Attacks in SDN Environments." *Future Generation Computer Systems*125 (2021): 156–167.
34. J. D. Gadze, A. A. Bamfo-Asante, J. O. Agyemang, H. Nunoo-Mensah,, and K. A. Opare. "An Investigation into the Application of Deep Learning in the Detection and Mitigation of DDoS Attack on SDN Controllers." *Technologies* 9, no. 1 (2021): 14.
35. I. Priyadarshini, P. Mohanty, A. Alkhayyat, R. Sharma,, and S. Kumar. "SDN and Application Layer DDoS Attacks Detection in IoT Devices by Attention-Based Bi-LSTM-CNN." *Transactions on Emerging Telecommunications Technologies* 34, no. 11 (2023): e4758.
36. Nawar Shaheed, Jumaah,, and Asghar Tajoddin Ashkafaki. "Hybrid Ensemble Deep Learning Framework for Efficient DDoS Attack Detection in Software-Defined Networks." *Journal of Electrical Systems* 20, no. 10s (2024): 6552–6561.
37. H. Min, Z. Yang, H. Sun,, and P. Zhang. 2023, December). "DDoS Attack Detection Model Based on CNN-LSTM." In *Fourth International Conference on Signal Processing and Computer Science (SPCS 2023)*, Vol 12970, 1014–1018. SPIE.
38. H. Polat, M. Türkoğlu, O. Polat,, and A. Şengür. "A Novel Approach for Accurate Detection of the DDoS Attacks in SDN-Based SCADA Systems Based on Deep Recurrent Neural Networks." *Expert Systems with Applications*197 (2022): 116748.
39. H. Elubeyd, and D. Yiltas-Kaplan. "Hybrid Deep Learning Approach for Automatic DoS/DDoS Attacks Detection in Software-Defined Networks." *Applied Sciences* 13, no. 6 (2023): 3828.
40. Beenish Habib, and Farida Khursheed. "Time-based DDoS attack detection through hybrid LSTM-CNN model architectures: An investigation of many-to-one and many-to-many approaches." *Concurrency and Computation: Practice and Experience* 36, no. 9 (2024): e7996.
41. N. U. Ain, M. Sardaraz, M. Tahir, M. W. Abo Elsoud,, and A. Alourani. "Securing IoT Networks Against DDoS Attacks: A Hybrid Deep Learning Approach." *Sensors* 25, no. 5 (2025): 1346.
42. Asha Varma Songa, and Ganesh Reddy Karri. "An integrated SDN framework for early detection of DDoS attacks in cloud computing." *Journal of Cloud Computing* 13, no. 1 (2024): 64.
43. A. Nazir, J. He, N. Zhu, S. S. Qureshi, S. U. Qureshi, F. Ullah, A. Wajahat,, and M. S. Pathan. "A Deep Learning-Based Novel Hybrid CNN-LSTM Architecture for Efficient Detection of Threats in the IoT Ecosystem." *Ain Shams Engineering Journal* 15, no. 7 (2024): 102777.
44. Thura Jabbar Khaleel, and Nadia Adnan Shiltagh. "DDoS attack detection using hybrid (CCN and LSTM) ML model." *Iraqi Journal for Computers and Informatics* 49, no. 2 (2023): 53–61.
45. Kun Wang, Yu Fu, Xueyuan Duan,, and Taotao Liu. "Detection and mitigation of DDoS attacks based on multi-dimensional characteristics in SDN." *Scientific Reports* 14, no. 1 (2024): 16421.
46. Vanlalruata Hnamte, Ashfaq Ahmad Najar, Hong Nhung-Nguyen, Jamal Hussain,, and Manohar Naik Sugali. "DDoS attack detection and mitigation using deep neural network in SDN environment." *Computers & Security*138 (2024): 103661.
47. Usham Sanjota Chanu, Khundrakpam Johnson Singh,, and Yambem Jina Chanu. "A dynamic feature selection technique to detect DDoS attack." *Journal of Information Security and Applications*74 (2023): 103445.
48. Nitin Rane, Saurabh Choudhary,, and Jayesh Rane. "Machine learning and deep learning: A comprehensive review on methods, techniques, applications, challenges, and future directions." . *Techniques, Applications, Challenges, and Future Directions*.
49. Marcos VO de Assis, Luiz F Carvalho, Joel JPC Rodrigues, Jaime Lloret,, and Mario L Proença Jr. "Near real-time security system applied to SDN environments in IoT networks using convolutional neural network." *Computers & Electrical Engineering*86 (2020): 106738.
50. Basheer Husham Ali, Nasri Sulaiman, SAR Al-Haddad, Rodziah Atan,, and Siti Lailatul Mohd Hassan. 2022. "DDos detection using active and idle features of revised cicflowmeter and statistical approaches." In *2022 4th International Conference on Advanced Science and Engineering (ICOASE)*, 148–153. IEEE.
51. Kai Mei, Meifang Tan, Zhihui Yang,, and Shaoyue Shi. 2022. "Modeling of feature selection based on random forest algorithm and pearson correlation coefficient." In *Journal of Physics: Conference Series*, Vol 2219, 012046. IOP Publishing.
52. Ruikui Ma, Xuebin Chen,, and Ran Zhai. "A DDoS attack detection method based on natural selection of features and models." *Electronics* 12, no. 4 (2023): 1059.
53. Mousa Alalhareth, and Sung-Chul Hong. "An improved mutual information feature selection technique for intrusion detection systems in the Internet of Medical Things." *Sensors* 23, no. 10 (2023): 4971.
54. Emad Ul Haq Qazi, Abdulrazaq Almorjan,, and Tanveer Zia. "A one-dimensional convolutional neural network (1D-CNN) based deep learning system for network intrusion detection." *Applied Sciences* 12, no. 16 (2022): 7986.
55. Jayanth Koushik. "Understanding convolutional neural networks." . *arXiv preprint arXiv:1605.09081*.
56. Ilaria Cacciari, and Anedio Ranfagni. "Hands-on Fundamentals of 1D Convolutional Neural Networks—A Tutorial for Beginner Users." *Applied Sciences (2076-3417)* 14, no. 18.
57. Ivan Malashin, Vadim Tynchenko, Andrei Gantimurov, Vladimir Nelyub,, and Aleksei Borodulin. "Applications of Long Short-Term Memory (LSTM) Networks in Polymeric Sciences: A Review." *Polymers* 16, no. 18 (2024): 2607.
58. Shiv Ram Dubey, Satish Kumar Singh,, and Bidyut Baran Chaudhuri. "Activation functions in deep learning: A comprehensive survey and benchmark." *Neurocomputing*503 (2022): 92–108.
59. Diederik P. Kingma, and Jimmy Ba. "Adam: A Method for Stochastic Optimization." . *arXiv preprint arXiv:1412.6980*.
60. Franco Jaraba, Gautam Mahajan, Jay Jani, Robert Ipu,, and Sergey Butakov. "Exploring current solutions against DDoS attacks in SDN environment." *Procedia Computer Science*238 (2024): 127–134.
61. Tapadhir Das, Osama Abu Hamdan, Shamik Sengupta,, and Engin Arslan. 2022. "Flood control: Tcp-syn flood detection for software-Defined Networks using openflow port statistics." In *2022 IEEE International Conference on Cyber Security and Resilience (CSR)*, 1–8. IEEE.
62. Mahmoud Said Elsayed, Nhien-An Le-Khac,, and Anca Delia Jurcut. "InSDN: A Novel SDN Intrusion Dataset." *IEEE Access*8 (2020): 165263–165284.
63. Dhruvkumar Dholakiya, Tanmay Kshirsagar,, and Amit Nayak. "Survey of mininet challenges, opportunities, and application in software-defined network (SDN)." . *Information and Communication Technology for Intelligent Systems: Proceedings of ICTIS 2020, Volume 2* (2021): 213–221.

64. Altuğ Çil, and Mehmet Demirci. "A comparative analysis of software-defined network controllers in terms of network forensics processes and capabilities." *Sigma Journal of Engineering and Natural Sciences* 42, no. 2 (2024): 425–437.
65. Achmad Basuki, Kasyful Amron,, and Primantara Hari Trisnawan. "Penerapan Dynamic Flow Removal untuk Mencegah Flow Table Overflow pada Software-Defined Networking." *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK)* 9, no. 6.
66. sFlow.org 2024. "sFlow Protocol." https://sflow.org/, Accessed: 2025-05-24.
67. D Narmadha et al. 2024. "Detection of distributed denial of service attack on controller in software defined network." In *2024 IEEE International Conference on Smart Power Control and Renewable Energy (ICSPCRE)*, 1–6. IEEE.
68. Kamal Saluja, Susama Bagchi, Vikas Solanki, Muhammad Numan Ali Khan, Esha Dhamija,, and Sanjoy Kumar Debnath. 2024. "Exploring robust ddos detection: A machine learning analysis with the cicddos2019 dataset." In *2024 IEEE 5th India Council International Subsections Conference (INDISCON)*, 1–6. IEEE.
69. Abdulmuneem Bashaiwth, Hamad Binsalleeh,, and Basil AsSadhan. "An explanation of the LSTM model used for DDoS attacks classification." *Applied Sciences* 13, no. 15 (2023): 8820.
70. Amreen Anbar 2023. "Classification of DDoS Attack with Machine Learning Architectures and Exploratory Analysis." MS thesis, The University of Western Ontario (Canada).
71. Caroline Panggabean, Chandrasekar Venkatachalam, Priyanka Shah, Sincy John, P Renuka Devi,, and Shanmugavalli Venkatachalam. 2024. "Intelligent DoS and DDoS Detection: A Hybrid GRU-NTM Approach to Network Security." In *2024 5th International Conference on Smart Electronics and Communication (ICOSEC)*, 658–665. IEEE.
72. Christian Czosseck, Rain Ottis,, and Anna-Maria Talihärm. "Estonia after the 2007 cyber attacks: Legal, strategic and organisational changes in cyber security." *International Journal of Cyber Warfare and Terrorism (IJCWT)* 1, no. 1 (2011): 24–34.
73. David Gillman, Yin Lin, Bruce Maggs,, and Ramesh K Sitaraman. "Protecting websites from attack with secure delivery networks." *Computer* 48, no. 4 (2015): 26–34.
74. Manos Antonakakis, Tim April, Michael Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J Alex Halderman, Luca Invernizzi, Michalis Kallitsis, et al. 2017. "Understanding the Mirai botnet." In *26th USENIX security symposium (USENIX Security 17)*, 1093–1110.
75. NETSCOUT Systems Inc. 2021. "2021 Threat Intelligence Report." , NETSCOUT. Accessed: 2025-05-07.
76. Hanan Sharif, Sardar Usman, Muhammad Hasnain, et al. "A machine learning based approach for the detection of DDoS attacks on internet of things using CICDDoS2019 dataset-PortMap." *Lahore Garrison University research journal of computer science and information technology* 8, no. 2.

Supporting Information

Additional supporting information can be found online in the Supporting Information section.